pi05 复现+微调

一、pi05 复现

官方源码链接：https://github.com/Physical-Intelligence/openpi

一、安装服务端（服务端用于执行机器人机械臂的动作，接收的输入是仿真环境的输入）

```Bash
git clone --recurse-submodules git@github.com:Physical-Intelligence/openpi.git
```

容易出现以下问题，是因为你的本地机器没有配置有效的 SSH 密钥，导致 GitHub 服务器拒绝了你的身份验证请求。

```
(base) ubuntu2025@uroot:~$ git clone --recurse-submodules git@github.com:Physical-Intelligence/openpi.git
正克隆到 'openpi'...
git@github.com: Permission denied (publickey).
fatal: 无法读取远程仓库。
```

步骤 1：检查本地是否已有 SSH 密钥

首先查看是否已存在 SSH 密钥（默认文件名通常是 `id_rsa` 或 `id_ed25519`）：

bash

```Bash
ls -la ~/.ssh
```

如果看到 `id_rsa` 和 `id_rsa.pub`（或 `id_ed25519` 和 `id_ed25519.pub`），说明已有密钥，直接跳到 步骤 3。

步骤 2：生成新的 SSH 密钥（如果没有）

如果没有密钥，执行以下命令生成（替换为你的 GitHub 绑定邮箱）：

bash

```
Bash
ssh-keygen -t ed25519 -C "your_email@example.com"
```

- 提示 "Enter a file in which to save the key" 时，直接按回车（使用默认路径）。
- 提示设置密码时，可直接按回车（无密码），或设置一个密码（后续使用时需要输入）。

步骤 3：将 SSH 公钥添加到 SSH 代理

确保 SSH 代理正在运行，并添加私钥：

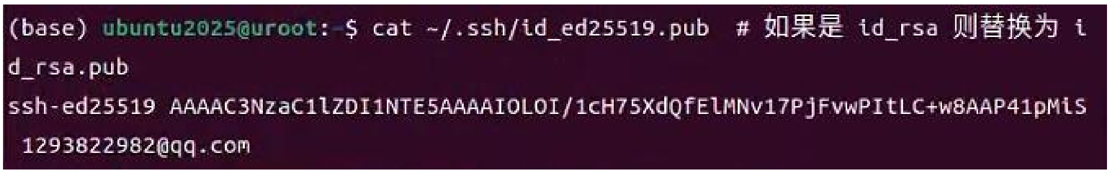
bash

```
Bash
# 启动代理 eval "$(ssh-agent -s)"# 添加私钥（如果是 id_rsa 则替换为 id_rsa）
ssh-add ~/.ssh/id_ed25519
```

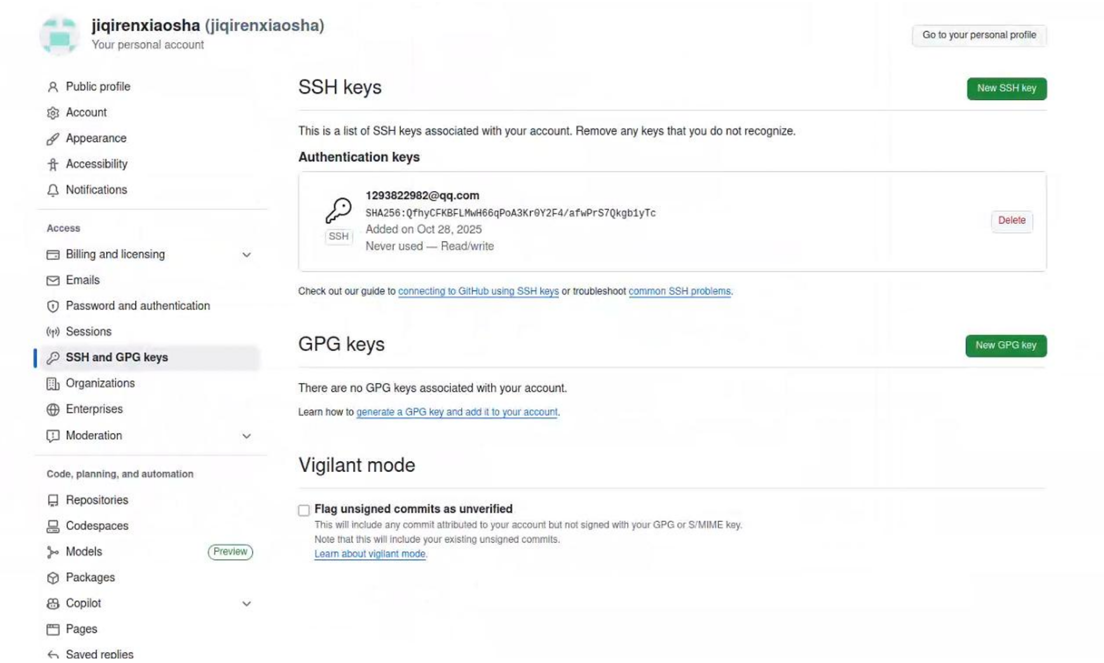
步骤 4：将公钥添加到你的 GitHub 账户

1. 查看公钥内容（复制输出的全部内容）：
2. bash

```
Bash
cat ~/.ssh/id_ed25519.pub  # 如果是 id_rsa 则替换为 id_rsa.pub
```



1. 登录 GitHub，进入 Settings（右上角头像 → Settings）。
2. 左侧菜单选择 SSH and GPG keys → 点击 New SSH key。
3. 在 "Title" 中输入一个标识（如你的机器名），在 "Key" 中粘贴刚才复制的公钥内容，点击 Add SSH key。



步骤 5：验证连接

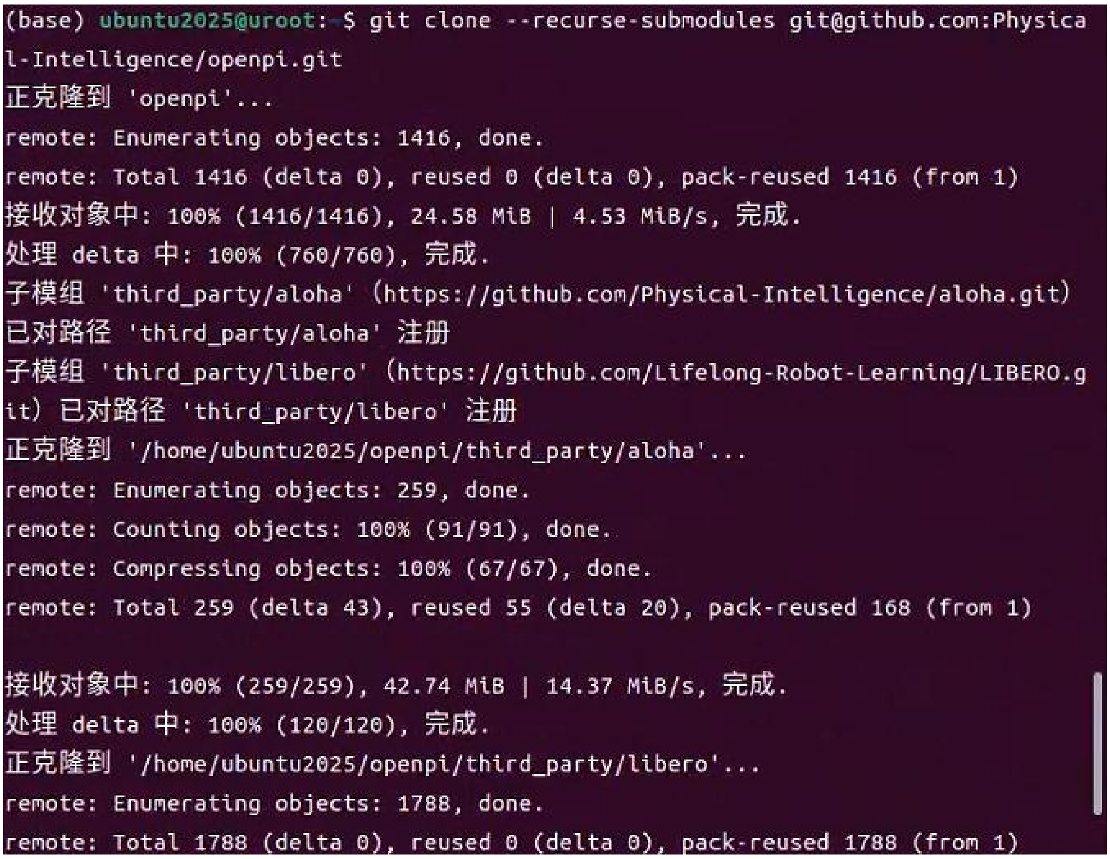
执行以下命令验证是否配置成功：

bash

```Bash
ssh -T git@github.com
```

如果看到类似 `Hi username! You've successfully authenticated...` 的提示，说明配置成功。

之后再执行 `git clone` 命令即可正常克隆仓库：



```
(base) ubuntu2025@uroot:~$ git clone --recurse-submodules git@github.com:Physica
l-Intelligence/openpi.git
正克隆到 'openpi'...
remote: Enumerating objects: 1416, done.
remote: Total 1416 (delta 0), reused 0 (delta 0), pack-reused 1416 (from 1)
接收对象中: 100% (1416/1416), 24.58 MiB | 4.53 MiB/s, 完成.
处理 delta 中: 100% (760/760), 完成.
子模组 'third_party/aloha' (https://github.com/Physical-Intelligence/aloha.git)
已对路径 'third_party/aloha' 注册
子模组 'third_party/libero' (https://github.com/Lifelong-Robot-Learning/LIBERO.g
it) 已对路径 'third_party/libero' 注册
正克隆到 '/home/ubuntu2025/openpi/third_party/aloha'...
remote: Enumerating objects: 259, done.
remote: Counting objects: 100% (91/91), done.
remote: Compressing objects: 100% (67/67), done.
remote: Total 259 (delta 43), reused 55 (delta 20), pack-reused 168 (from 1)

接收对象中: 100% (259/259), 42.74 MiB | 14.37 MiB/s, 完成.
处理 delta 中: 100% (120/120), 完成.
正克隆到 '/home/ubuntu2025/openpi/third_party/libero'...
remote: Enumerating objects: 1788, done.
remote: Total 1788 (delta 0), reused 0 (delta 0), pack-reused 1788 (from 1)
```

二、安装 uv 包管理工具并拉取依赖包

```Plain Text
conda create -n uv_envs python=3.10
```

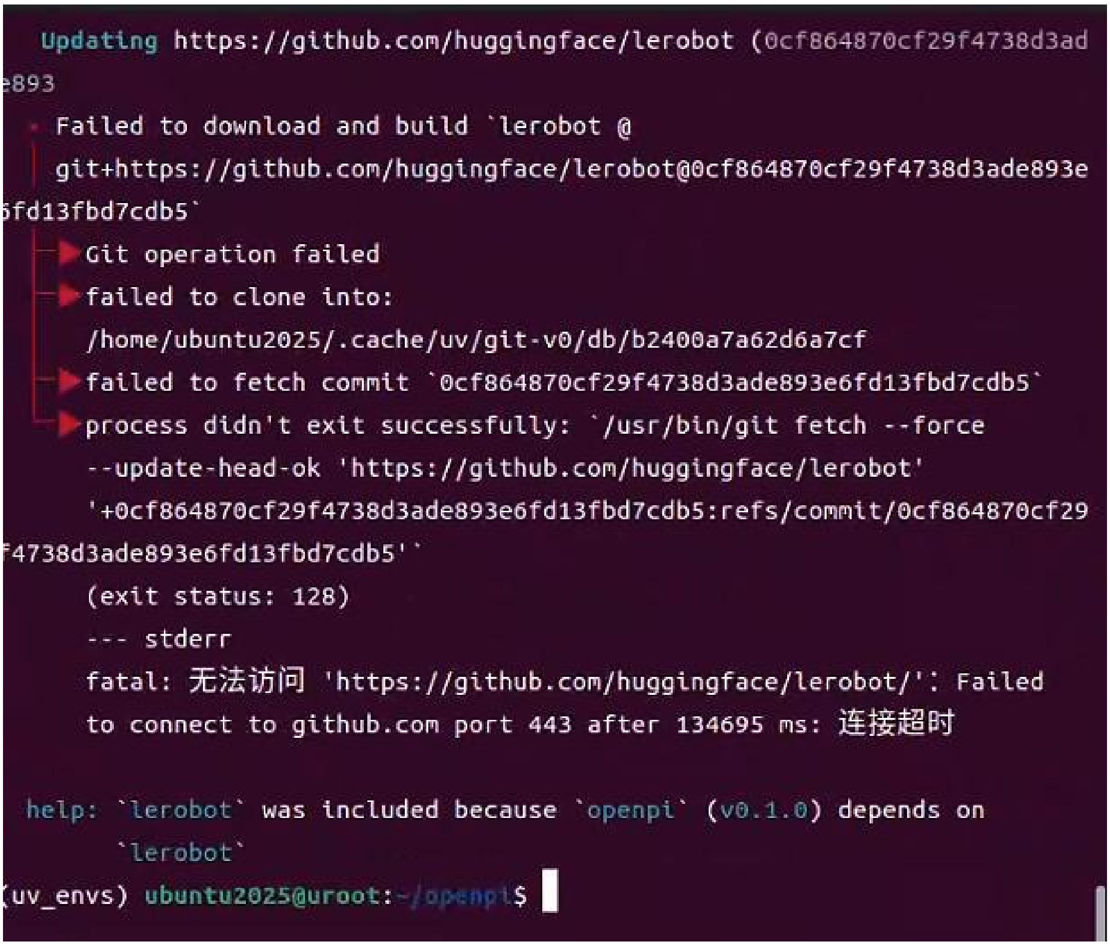
```Plain Text
conda activate uv_envs
```

```Plain Text
pip install uv
```

```Plain Text
cd openpi/ # 需要该目录下的：pyproject.toml
```

```Plain Text
GIT_LFS_SKIP_SMUDGE=1 uv sync # 根据pyproject.toml 下载依赖包
```

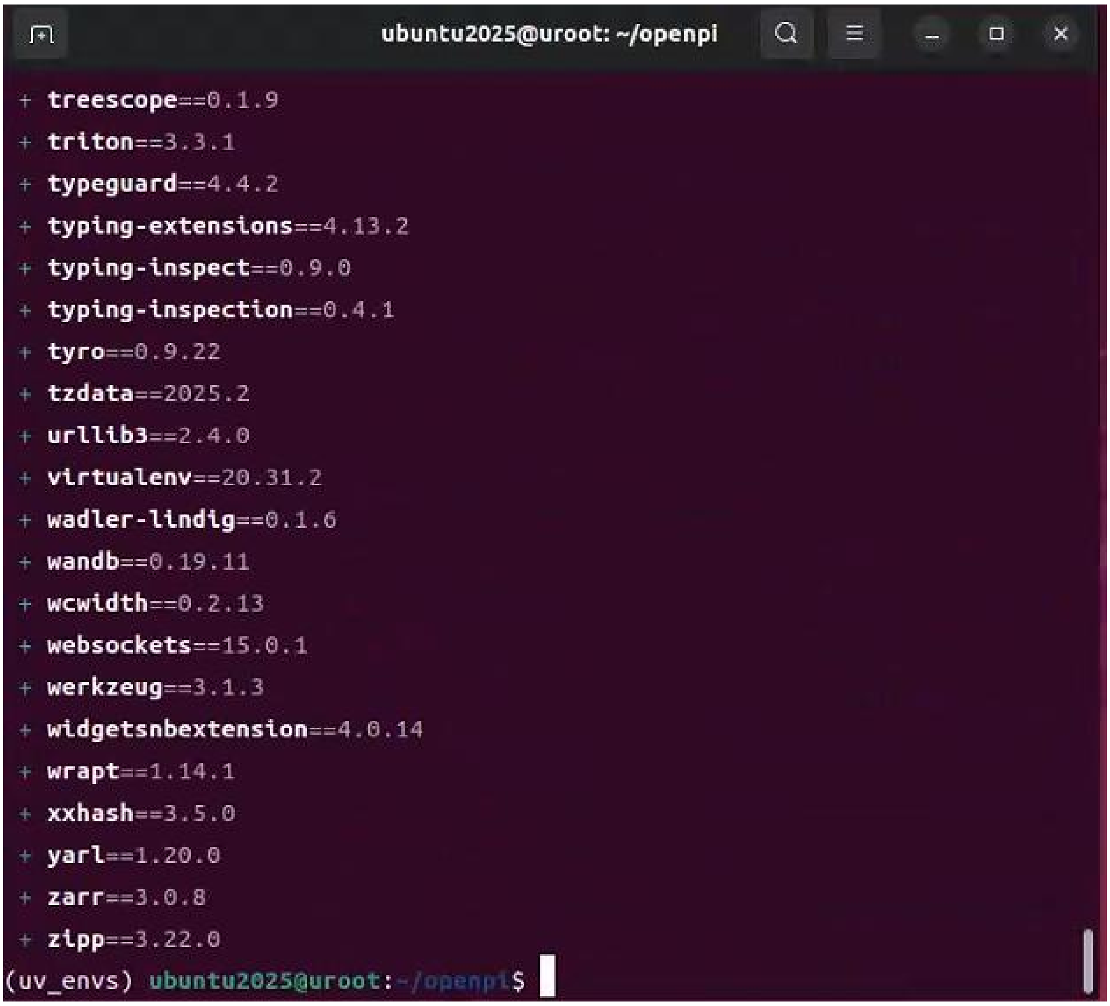
安装过程可能出现以下情况:



这个错误提示显示在下载 `lerobot` 依赖时连接 GitHub 超时（端口 443 连接失败），主要原因可能是网络问题或 GitHub 访问受限。一般是网络波动导致的，可以重新输入一次:

```Plain Text
GIT_LFS_SKIP_SMUDGE=1 uv sync
```

出现以下结果就是成功安装:



三、启动模型服务（server 端）

```
Plain Text
conda activate uv_envs
cd openpi
```

```
Plain Text
source ./.venv/bin/activate # 激活uv 的 Python 环境
```

```
Plain Text
export http_proxy=http://192.168.1.3:7890
```

```
Plain Text
export https_proxy=http://192.168.1.3:7890
```

```
Plain Text
uv run scripts/serve_policy.py --env LIBERO # 启动服务端监听
```

```
(uv_envs) ubuntu2025@uroot:~/openpi$ source ./.venv/bin/activate
(openpi) (uv_envs) ubuntu2025@uroot:~/openpi$ uv run scripts/serve_policy.
py --env LIBERO
warning: The `tool.uv.dev-dependencies` field (used in `packages/openpi-cl
ient/pyproject.toml`) is deprecated and will be removed in a future releas
e; use `dependency-groups.dev` instead

INFO:openpi.shared.download:Downloading gs://openpi-assets/checkpoints/pi0
5_libero to /home/ubuntu2025/.cache/openpi/openpi-assets/checkpoints/pi05_
libero
```

如果出现这个错误是由于无法连接到 `storage.googleapis.com`（Google Cloud Storage 服务）导致的，具体表现为端口 443 连接被重置，通常是网络环境限制了对 Google 服务的访问。这种情况就需要科学上网了或者手动下载：

```
Plain Text
sudo apt  install awscli
aws s3 cp s3://openpi-assets/checkpoints/pi0_base/assets ./ --
recursive --no-sign-request
aws s3 cp s3://openpi-assets/checkpoints/pi05_libero ./ --
recursive --no-sign-request
cp -r {上述下载文件的路径} ~/.cache/openpi
cp -r assets/ ~/.cache/openpi
```

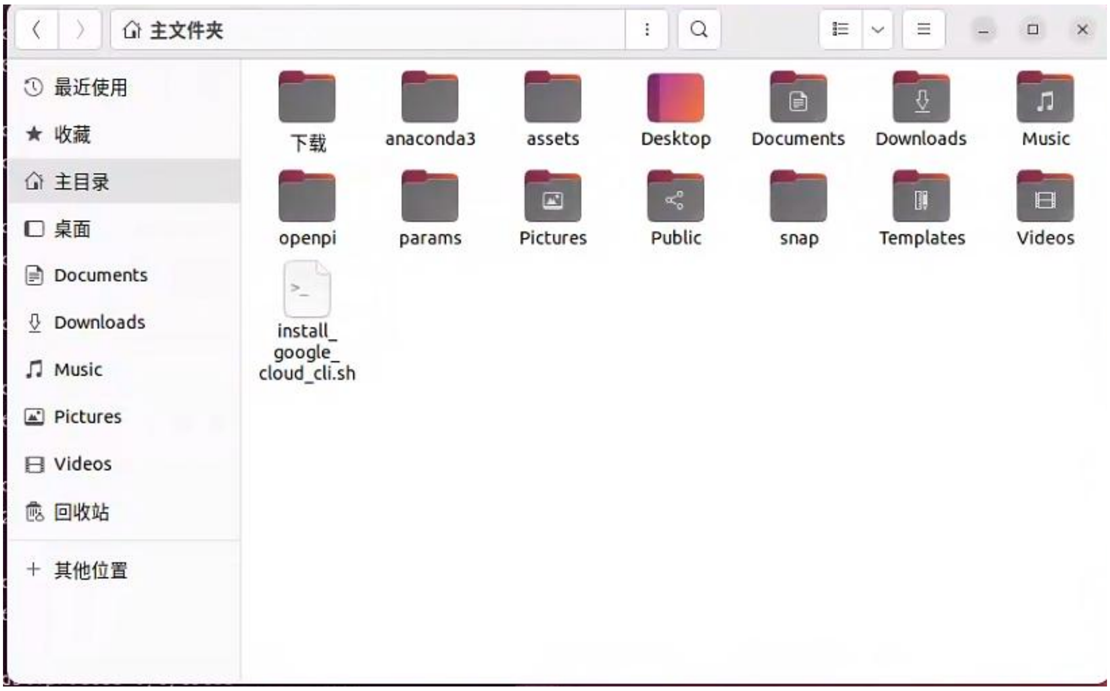
首先安装 awscli：

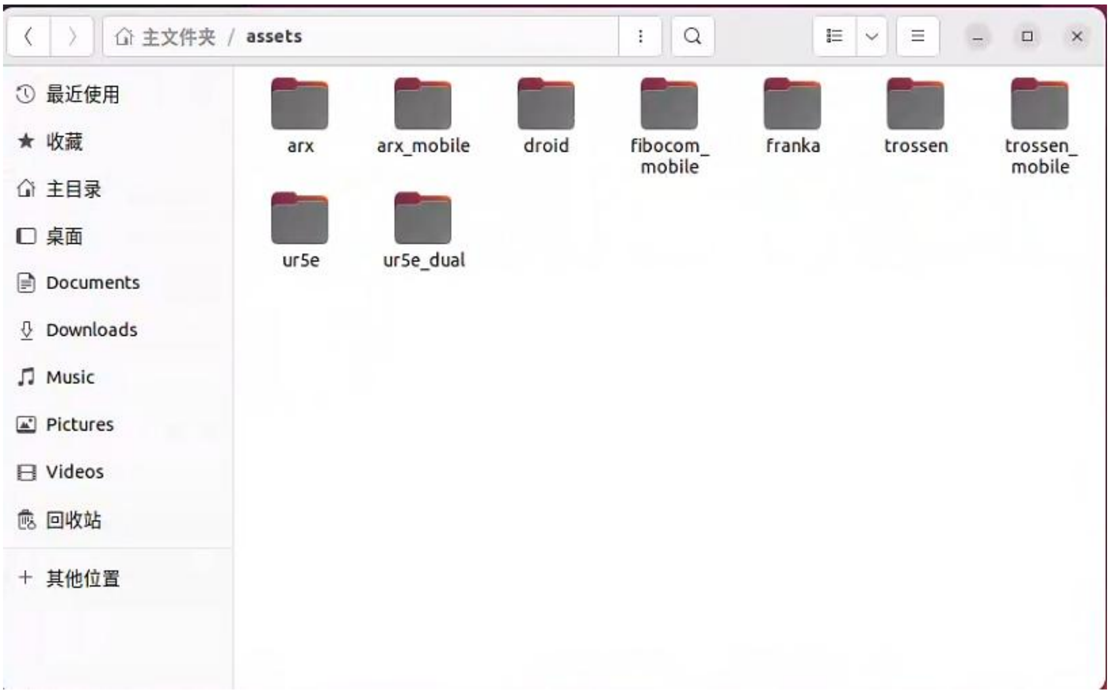
```
(openpi) (uv_envs) ubuntu2025@uroot:~/openpi$ sudo apt  install awscli
[sudo] ubuntu2025 的密码:
对不起，请重试。
[sudo] ubuntu2025 的密码:
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
将会同时安装下列软件:
  docutils-common groff gsfonts imagemagick imagemagick-6-common
  imagemagick-6.q16 libaom3 libdav1d5 libde265-0 libfftw3-double3
  libheif1 libilmbase25 libjxr-tools libjxr0 liblqr-1-0
  libmagickcore-6.q16-6 libmagickcore-6.q16-6-extra
  libmagickwand-6.q16-6 libnetpbm10 libopenexr25 libx265-199 netpbm
  psutils python3-botocore python3-docutils python3-jmespath
  python3-pyasn1 python3-pygments python3-roman python3-rsa
  python3-s3transfer
```

之后新开一个终端，进行手动下载 pi0_base 权重:

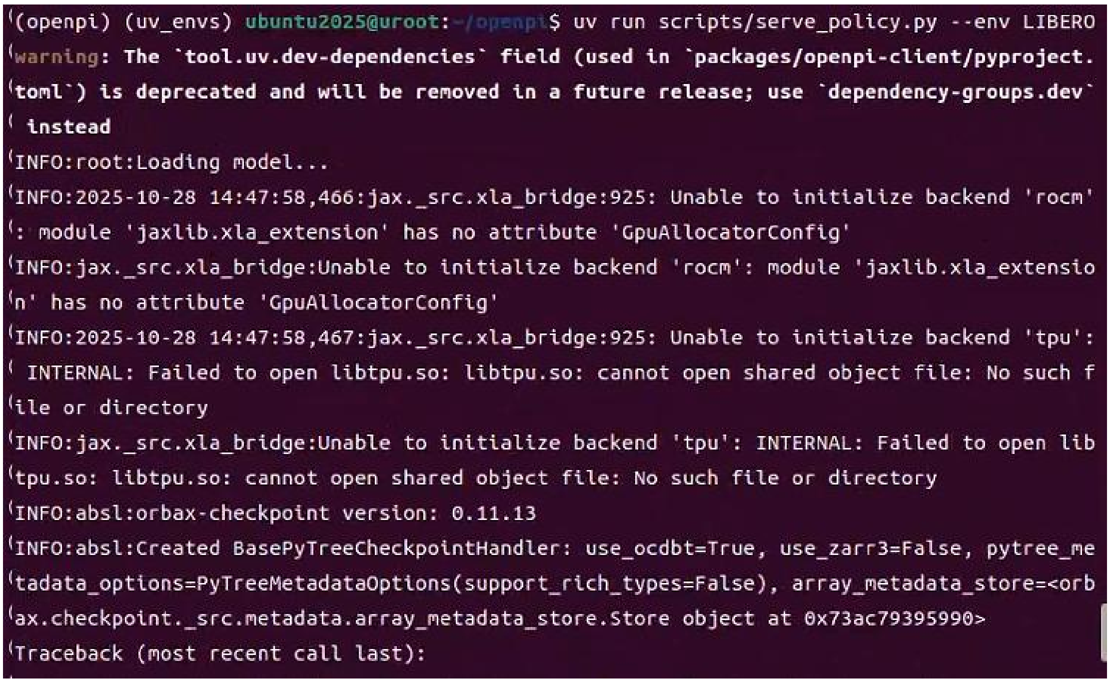
```
ubuntu2025@uroot: ~
(base) ubuntu2025@uroot:~$ ls
下载         Documents                    Music     Public     Videos
anaconda3    Downloads                    openpi    snap
Desktop      install_google_cloud_cli.sh  Pictures  Templates
(base) ubuntu2025@uroot:~$ aws s3 cp s3://openpi-assets/checkpoints/pi0_base ./
--recursive --no-sign-request
download: s3://openpi-assets/checkpoints/pi0_base/assets/droid/norm_stats.json t
o assets/droid/norm_stats.json
download: s3://openpi-assets/checkpoints/pi0_base/assets/trossen_mobile/norm_sta
ts.json to assets/trossen_mobile/norm_stats.json
download: s3://openpi-assets/checkpoints/pi0_base/assets/arx/norm_stats.json to
assets/arx/norm_stats.json
download: s3://openpi-assets/checkpoints/pi0_base/assets/arx_mobile/norm_stats.j
son to assets/arx_mobile/norm_stats.json
download: s3://openpi-assets/checkpoints/pi0_base/assets/trossen/norm_stats.json
 to assets/trossen/norm_stats.json
download: s3://openpi-assets/checkpoints/pi0_base/params/_CHECKPOINT_METADATA to
 params/_CHECKPOINT_METADATA
download: s3://openpi-assets/checkpoints/pi0_base/assets/franka/norm_stats.json
to assets/franka/norm_stats.json
download: s3://openpi-assets/checkpoints/pi0_base/assets/ur5e/norm_stats.json to
 assets/ur5e/norm_stats.json
download: s3://openpi-assets/checkpoints/pi0_base/params/_METADATA to params/_ME
TADATA
```

最后下载到我的主目录文件夹下，如图所示:


主文件夹
最近使用
收藏
主目录
桌面
Documents
Downloads
Music
Pictures
Videos
回收站
其他位置
下载
anaconda3
assets
Desktop
Documents
Downloads
Music
openpi
params
Pictures
Public
snap
Templates
Videos
install_google_cloud_cli.sh


主文件夹 / assets
最近使用
收藏
主目录
桌面
Documents
Downloads
Music
Pictures
Videos
回收站
其他位置
arx
arx_mobile
droid
fibocom_mobile
franka
trossen
trossen_mobile
ur5e
ur5e_dual

出现如下错误：



```
(openpi) (uv_envs) ubuntu2025@uroot:~/openpi$ uv run scripts/serve_policy.py --env LIBERO
warning: The `tool.uv.dev-dependencies` field (used in `packages/openpi-client/pyproject.
toml`) is deprecated and will be removed in a future release; use `dependency-groups.dev`
 instead
INFO:root:Loading model...
INFO:2025-10-28 14:47:58,466:jax._src.xla_bridge:925: Unable to initialize backend 'rocm'
: module 'jaxlib.xla_extension' has no attribute 'GpuAllocatorConfig'
INFO:jax._src.xla_bridge:Unable to initialize backend 'rocm': module 'jaxlib.xla_extensio
n' has no attribute 'GpuAllocatorConfig'
INFO:2025-10-28 14:47:58,467:jax._src.xla_bridge:925: Unable to initialize backend 'tpu':
 INTERNAL: Failed to open libtpu.so: libtpu.so: cannot open shared object file: No such f
ile or directory
INFO:jax._src.xla_bridge:Unable to initialize backend 'tpu': INTERNAL: Failed to open lib
tpu.so: libtpu.so: cannot open shared object file: No such file or directory
INFO:absl:orbax-checkpoint version: 0.11.13
INFO:absl:Created BasePyTreeCheckpointHandler: use_ocdbt=True, use_zarr3=False, pytree_me
tadata_options=PyTreeMetadataOptions(support_rich_types=False), array_metadata_store=<orb
ax.checkpoint._src.metadata.array_metadata_store.Store object at 0x73ac79395990>
Traceback (most recent call last):
```

错误提示 `FileNotFoundError: Metadata file (named _METADATA) does not exist` 表明程序在 `/home/ubuntu2025/.cache/openpi/openpi-assets/checkpoints/pi05_libero/params` 路径下找不到关键的元数据文件 `_METADATA`，这意味着 `pi05_libero` 模型的参数文件未正确下载或缺失。

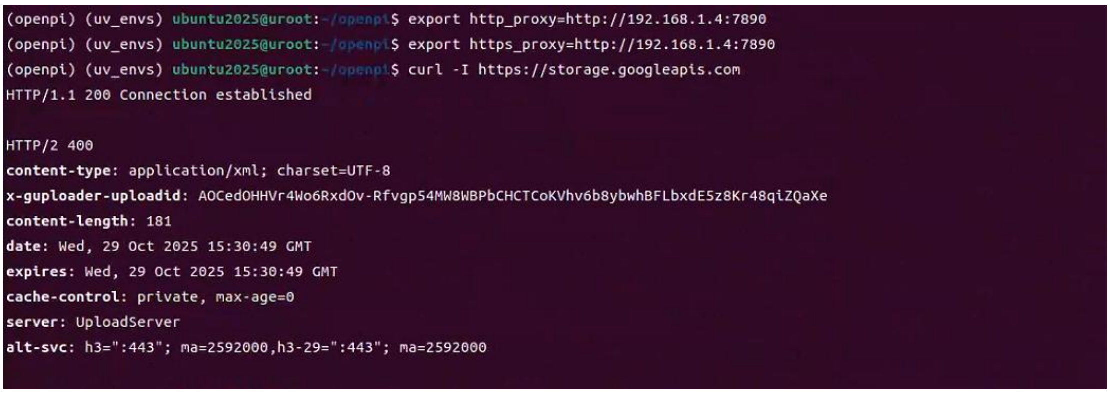
这时候需要局域网内 windows 系统电脑科学上网，Ubuntu 系统电脑可通过以下方式共享其网络(具体 ip 根据电脑查到的而定):

```Plain Text
export http_proxy=http://192.168.1.8:7890
```

```Plain Text
export https_proxy=http://192.168.1.8:7890
```

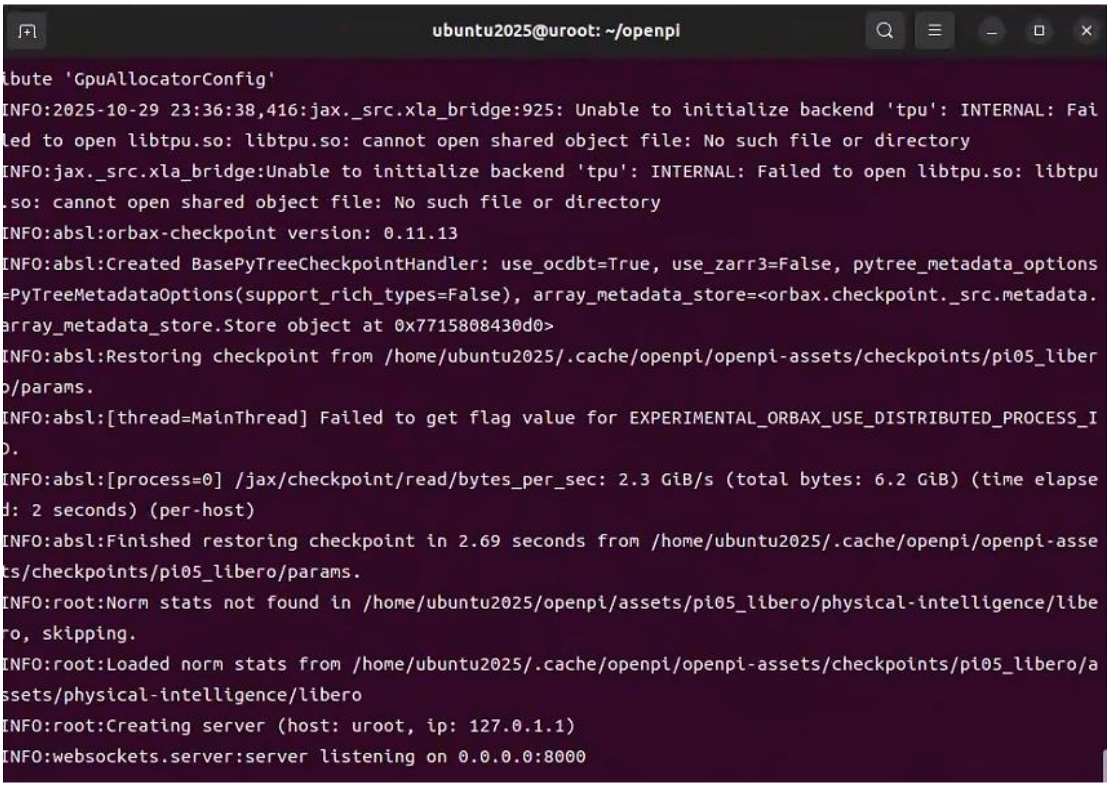
```Plain Text
curl -I https://storage.googleapis.com
```

出现下面情况就表示可以连上 googleapis.com:



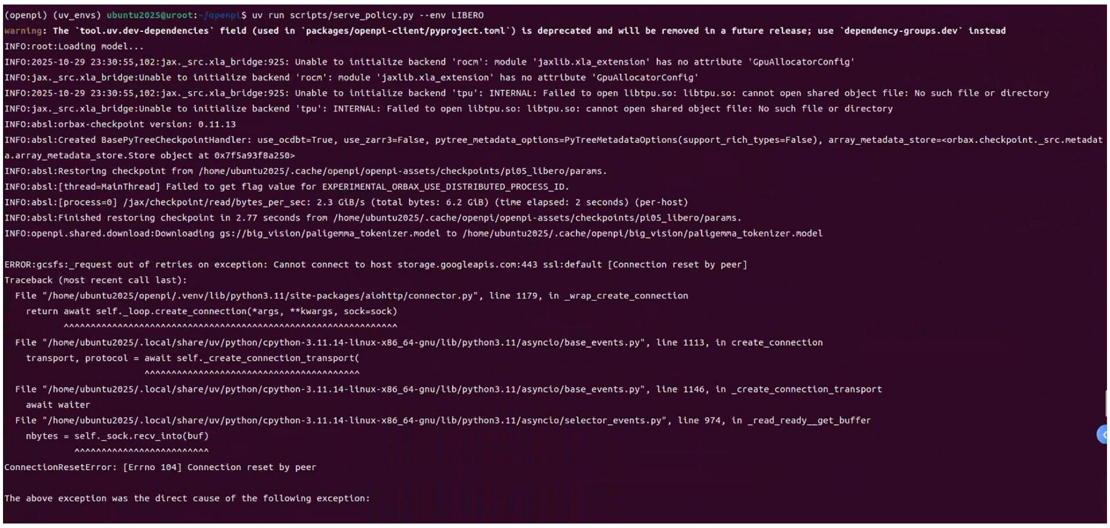
接下来就可以重新运行，出现下面情况就是连接成功，运行正常结束时会看见启动了一个 web 接口的监听, 如下图所示，不要关！！！ 这个启动的端口会一直运行，以等待客戶端发送请求。之所以要这样设计，是因为π0 采用了 CS 架构，在服务端（Server 端）进行pi0 模型的部署，客戶端（Client）负责传递 observation（包括摄像头输入，机器人的关节状态）给 server 端。

```Plain Text
uv run scripts/serve_policy.py --env LIBERO # 启动服务端监听
```



```
ubuntu2025@uroot: ~/openpi
ibute 'GpuAllocatorConfig'
INFO:2025-10-29 23:36:38,416:jax._src.xla_bridge:925: Unable to initialize backend 'tpu': INTERNAL: Failed to open libtpu.so: libtpu.so: cannot open shared object file: No such file or directory
INFO:jax._src.xla_bridge:Unable to initialize backend 'tpu': INTERNAL: Failed to open libtpu.so: libtpu.so: cannot open shared object file: No such file or directory
INFO:absl:orbax-checkpoint version: 0.11.13
INFO:absl:Created BasePyTreeCheckpointHandler: use_ocdbt=True, use_zarr3=False, pytree_metadata_options=PyTreeMetadataOptions(support_rich_types=False), array_metadata_store=<orbax.checkpoint._src.metadata.array_metadata_store.Store object at 0x7715808430d0>
INFO:absl:Restoring checkpoint from /home/ubuntu2025/.cache/openpi/openpi-assets/checkpoints/pi05_libero/params.
INFO:absl:[thread=MainThread] Failed to get flag value for EXPERIMENTAL_ORBAX_USE_DISTRIBUTED_PROCESS_ID.
INFO:absl:[process=0] /jax/checkpoint/read/bytes_per_sec: 2.3 GiB/s (total bytes: 6.2 GiB) (time elapsed: 2 seconds) (per-host)
INFO:absl:Finished restoring checkpoint in 2.69 seconds from /home/ubuntu2025/.cache/openpi/openpi-assets/checkpoints/pi05_libero/params.
INFO:root:Norm stats not found in /home/ubuntu2025/openpi/assets/pi05_libero/physical-intelligence/libero, skipping.
INFO:root:Loaded norm stats from /home/ubuntu2025/.cache/openpi/openpi-assets/checkpoints/pi05_libero/assets/physical-intelligence/libero
INFO:root:Creating server (host: uroot, ip: 127.0.1.1)
INFO:websockets.server:server listening on 0.0.0.0:8000
```

可能出现的错误：是由于程序无法连接到 `storage.googleapis.com` 下载 `paligemma_tokenizer.model`（路径 `gs://big_vision/paligemma_tokenizer.model`），本质还是网络无法访问 Google 服务导致的。



```
(openpi) (uv_envs) ubuntu2025@uroot:~/openpi$ uv run scripts/serve_policy.py --env LIBERO
warning: The `tool.uv.dev-dependencies` field (used in `packages/openpi-client/pyproject.toml`) is deprecated and will be removed in a future release; use `dependency-groups.dev` instead
INFO:root:Loading model...
INFO:2025-10-29 23:30:55,102:jax._src.xla_bridge:925: Unable to initialize backend 'rocm': module 'jaxlib.xla_extension' has no attribute 'GpuAllocatorConfig'
INFO:jax._src.xla_bridge:Unable to initialize backend 'rocm': module 'jaxlib.xla_extension' has no attribute 'GpuAllocatorConfig'
INFO:2025-10-29 23:30:55,102:jax._src.xla_bridge:925: Unable to initialize backend 'tpu': INTERNAL: Failed to open libtpu.so: libtpu.so: cannot open shared object file: No such file or directory
INFO:jax._src.xla_bridge:Unable to initialize backend 'tpu': INTERNAL: Failed to open libtpu.so: libtpu.so: cannot open shared object file: No such file or directory
INFO:absl:orbax-checkpoint version: 0.11.13
INFO:absl:Created BasePyTreeCheckpointHandler: use_ocdbt=True, use_zarr3=False, pytree_metadata_options=PyTreeMetadataOptions(support_rich_types=False), array_metadata_store=<orbax.checkpoint._src.metadata.array_metadata_store.Store object at 0x7f5a93f8a250>
INFO:absl:Restoring checkpoint from /home/ubuntu2025/.cache/openpi/openpi-assets/checkpoints/pi05_libero/params.
INFO:absl:[thread=MainThread] Failed to get flag value for EXPERIMENTAL_ORBAX_USE_DISTRIBUTED_PROCESS_ID.
INFO:absl:[process=0] /jax/checkpoint/read/bytes_per_sec: 2.3 GiB/s (total bytes: 6.2 GiB) (time elapsed: 2 seconds) (per-host)
INFO:absl:Finished restoring checkpoint in 2.77 seconds from /home/ubuntu2025/.cache/openpi/openpi-assets/checkpoints/pi05_libero/params.
INFO:openpi.shared.download:Downloading gs://big_vision/paligemma_tokenizer.model to /home/ubuntu2025/.cache/openpi/big_vision/paligemma_tokenizer.model

ERROR:gcsfs:_request out of retries on exception: Cannot connect to host storage.googleapis.com:443 ssl:default [Connection reset by peer]
Traceback (most recent call last):
  File "/home/ubuntu2025/openpi/.venv/lib/python3.11/site-packages/aiohttp/connector.py", line 1179, in _wrap_create_connection
    return await self._loop.create_connection(*args, **kwargs, sock=sock)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ubuntu2025/.local/share/uv/python/cpython-3.11.14-linux-x86_64-gnu/lib/python3.11/asyncio/base_events.py", line 1113, in create_connection
    transport, protocol = await self._create_connection_transport(
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ubuntu2025/.local/share/uv/python/cpython-3.11.14-linux-x86_64-gnu/lib/python3.11/asyncio/base_events.py", line 1146, in _create_connection_transport
    await waiter
  File "/home/ubuntu2025/.local/share/uv/python/cpython-3.11.14-linux-x86_64-gnu/lib/python3.11/asyncio/selector_events.py", line 974, in _read_ready__get_buffer
    nbytes = self._sock.recv_into(buf)
             ^^^^^^^^^^^^^^^^^^^^^^^^^
ConnectionResetError: [Errno 104] Connection reset by peer

The above exception was the direct cause of the following exception:
```

解决办法科学上网用 `wget` 下载：：

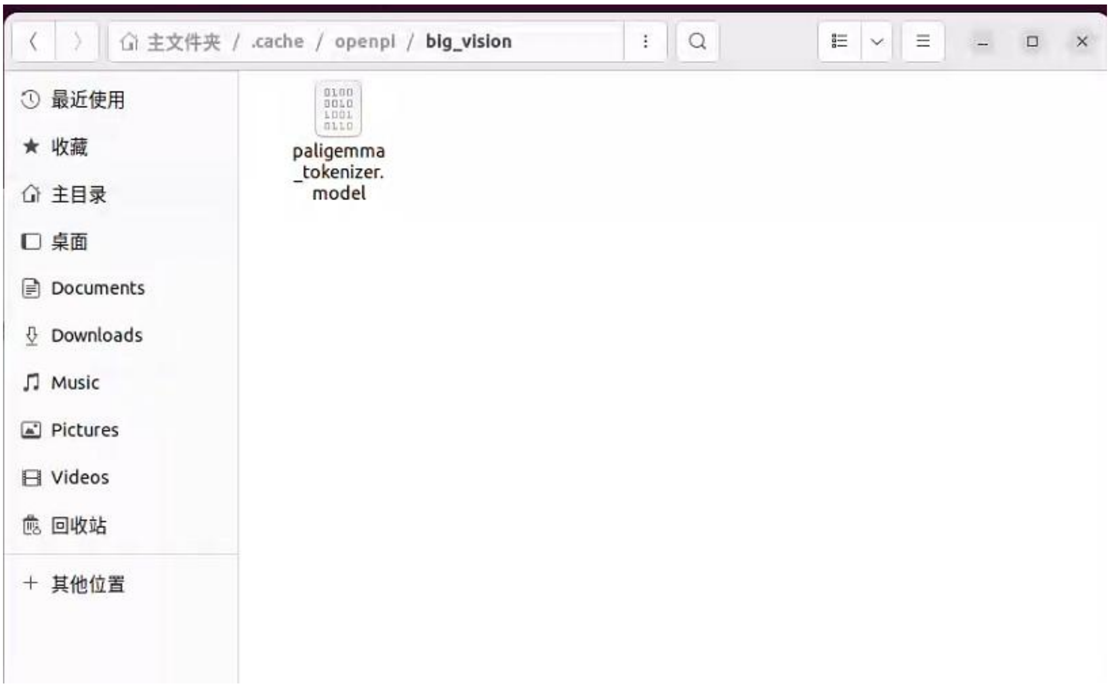
```Plain Text
export http_proxy=http://192.168.1.5:7890
```

```Plain Text
```

```
export https_proxy=http://192.168.1.5:7890
```

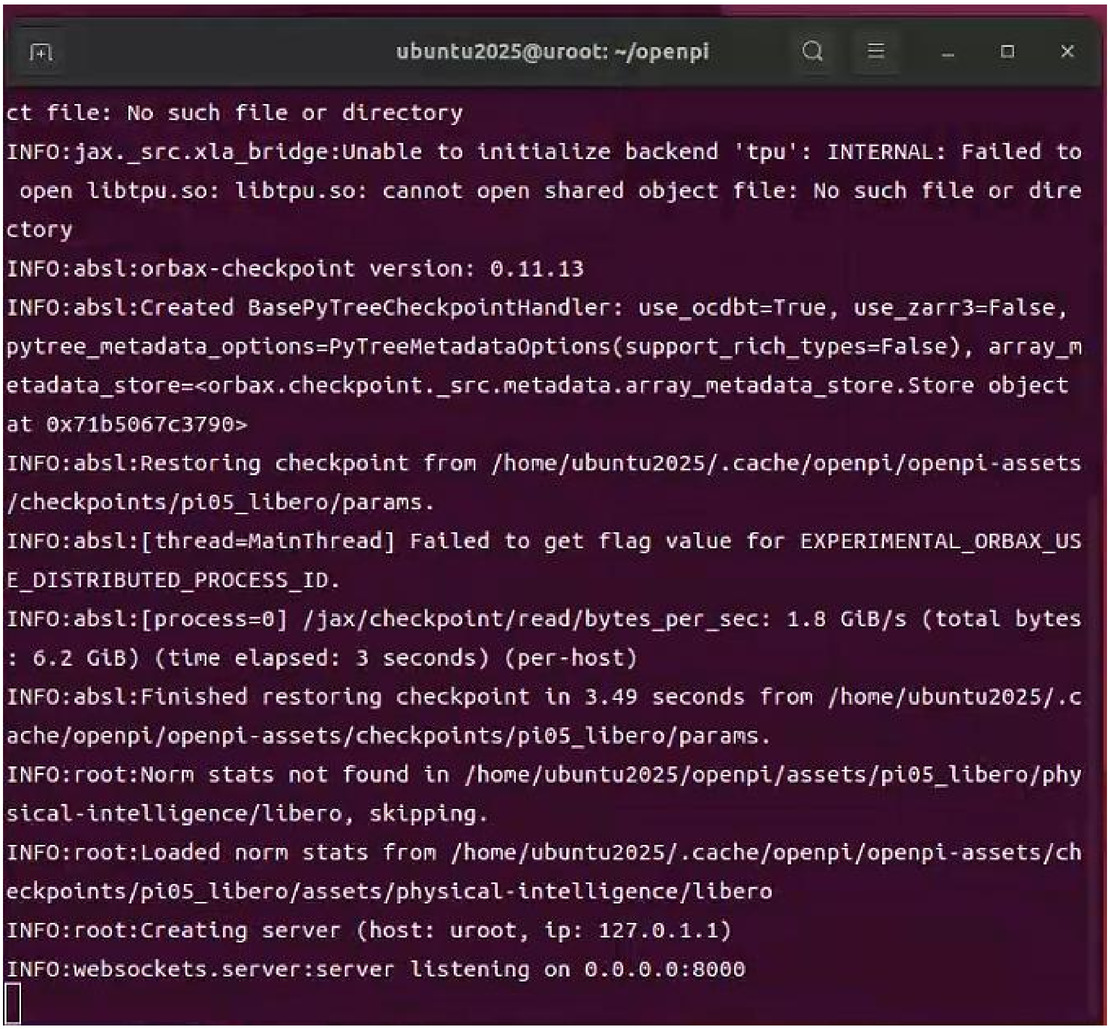
```Plain Text
wget
https://storage.googleapis.com/big_vision/paligemma_tokenizer.mode
l -O paligemma_tokenizer.model
```

最后将下载后的 model 文件放置到下面位置再次重新运行下述代码即可：




```Plain Text
uv run scripts/serve_policy.py --env LIBERO
```

命令执行成功后不要关闭该终端，再开一个新终端完成后续操作。



可能出现代理环境变量已经设置成功（你之前 `uv run python -c ...` 能看到 proxy）。

但 gcsfs/aiohttp 这条链路仍然在“直连”的报错：需要将自己对应目录下的这个文件替换成下面/home/ubuntu2025/openpi/src/openpi/shared/download.py

```python
import concurrent.futures
import datetime
import logging
import os
import pathlib
import re
import shutil
import stat
import time
import urllib.parse

import filelock
```

```
import fsspec
import fsspec.generic
import tqdm_loggable.auto as tqdm

# Environment variable to control cache directory path,
~/.cache/openpi will be used by default.
_OPENPI_DATA_HOME = "OPENPI_DATA_HOME"
DEFAULT_CACHE_DIR = "~/.cache/openpi"

logger = logging.getLogger(__name__)


def get_cache_dir() -> pathlib.Path:
    cache_dir = pathlib.Path(os.getenv(_OPENPI_DATA_HOME,
DEFAULT_CACHE_DIR)).expanduser().resolve()
    cache_dir.mkdir(parents=True, exist_ok=True)
    _set_folder_permission(cache_dir)
    return cache_dir


def maybe_download(url: str, *, force_download: bool = False,
**kwargs) -> pathlib.Path:
    """Download a file or directory from a remote filesystem to
the local cache, and return the local path.

    If the local file already exists, it will be returned
directly.

    It is safe to call this function concurrently from multiple
processes.
    See `get_cache_dir` for more details on the cache directory.

    Args:
        url: URL to the file to download.
        force_download: If True, the file will be downloaded even
if it already exists in the cache.
        **kwargs: Additional arguments to pass to fsspec.

    Returns:
        Local path to the downloaded file or directory. That path
is guaranteed to exist and is absolute.
    """
    # Don't use fsspec to parse the url to avoid unnecessary
connection to the remote filesystem.
```

```
    parsed = urllib.parse.urlparse(url)

    # Short circuit if this is a local path.
    if parsed.scheme == "":
        path = pathlib.Path(url)
        if not path.exists():
            raise FileNotFoundError(f"File not found at {url}")
        return path.resolve()

    cache_dir = get_cache_dir()

    local_path = cache_dir / parsed.netloc /
parsed.path.strip("/")
    local_path = local_path.resolve()

    # Check if the cache should be invalidated.
    invalidate_cache = False
    if local_path.exists():
        if force_download or _should_invalidate_cache(cache_dir,
local_path):
            invalidate_cache = True
        else:
            return local_path

    try:
        lock_path = local_path.with_suffix(".lock")
        with filelock.FileLock(lock_path):
            # Ensure consistent permissions for the lock file.
            _ensure_permissions(lock_path)
            # First, remove the existing cache if it is expired.
            if invalidate_cache:
                logger.info(f"Removing expired cached entry:
{local_path}")
                if local_path.is_dir():
                    shutil.rmtree(local_path)
                else:
                    local_path.unlink()

            # Download the data to a local cache.
            logger.info(f"Downloading {url} to {local_path}")
            scratch_path = local_path.with_suffix(".partial")
            _download_fsspec(url, scratch_path, **kwargs)

            shutil.move(scratch_path, local_path)
```

```
            _ensure_permissions(local_path)

    except PermissionError as e:
        msg = (
            f"Local file permission error was encountered while
downloading {url}. "
            f"Please try again after removing the cached data
using: `rm -rf {local_path}*`"
        )
        raise PermissionError(msg) from e

    return local_path


def _download_fsspec(url: str, local_path: pathlib.Path, **kwargs)
-> None:
    """Download a file from a remote filesystem to the local
cache, and return the local path."""

    # ===================== [FIX] ensure gcsfs/aiohttp respects
HTTP(S)_PROXY =====================
    # gcsfs uses aiohttp under the hood; aiohttp only reads env
proxies when trust_env=True.
    parsed = urllib.parse.urlparse(url)
    if parsed.scheme in ("gs", "gcs"):
        session_kwargs = kwargs.get("session_kwargs") or {}
        # make a copy to avoid mutating caller's dict object
unexpectedly
        session_kwargs = dict(session_kwargs)
        session_kwargs.setdefault("trust_env", True)
        kwargs["session_kwargs"] = session_kwargs
    #
==================================================================
==========================

    fs, _ = fsspec.core.url_to_fs(url, **kwargs)
    info = fs.info(url)
    # Folders are represented by 0-byte objects with a trailing
forward slash.
    if is_dir := (info["type"] == "directory" or (info["size"] ==
0 and info["name"].endswith("/"))):
        total_size = fs.du(url)
    else:
        total_size = info["size"]
```

```
    with tqdm.tqdm(total=total_size, unit="iB", unit_scale=True,
unit_divisor=1024) as pbar:
        executor =
concurrent.futures.ThreadPoolExecutor(max_workers=1)
        future = executor.submit(fs.get, url, local_path,
recursive=is_dir)
        while not future.done():
            current_size = sum(f.stat().st_size for f in
[*local_path.rglob("*"), local_path] if f.is_file())
            pbar.update(current_size - pbar.n)
            time.sleep(1)
        pbar.update(total_size - pbar.n)


def _set_permission(path: pathlib.Path, target_permission: int):
    """chmod requires executable permission to be set, so we skip
if the permission is already match with the target."""
    if path.stat().st_mode & target_permission ==
target_permission:
        logger.debug(f"Skipping {path} because it already has
correct permissions")
        return
    path.chmod(target_permission)
    logger.debug(f"Set {path} to {target_permission}")


def _set_folder_permission(folder_path: pathlib.Path) -> None:
    """Set folder permission to be read, write and searchable."""
    _set_permission(folder_path, stat.S_IRWXU | stat.S_IRWXG |
stat.S_IRWXO)


def _ensure_permissions(path: pathlib.Path) -> None:
    """Since we are sharing cache directory with containerized
runtime as well as training script, we need to
    ensure that the cache directory has the correct permissions.
    """

    def _setup_folder_permission_between_cache_dir_and_path(path:
pathlib.Path) -> None:
        cache_dir = get_cache_dir()
        relative_path = path.relative_to(cache_dir)
        moving_path = cache_dir
        for part in relative_path.parts:
```

```
            _set_folder_permission(moving_path / part)
            moving_path = moving_path / part

    def _set_file_permission(file_path: pathlib.Path) -> None:
        """Set all files to be read & writable, if it is a script,
keep it as a script."""
        file_rw = stat.S_IRUSR | stat.S_IWUSR | stat.S_IRGRP |
stat.S_IWGRP | stat.S_IROTH | stat.S_IWOTH
        if file_path.stat().st_mode & 0o100:
            _set_permission(file_path, file_rw | stat.S_IXUSR |
stat.S_IXGRP | stat.S_IXOTH)
        else:
            _set_permission(file_path, file_rw)

    _setup_folder_permission_between_cache_dir_and_path(path)
    for root, dirs, files in os.walk(str(path)):
        root_path = pathlib.Path(root)
        for file in files:
            file_path = root_path / file
            _set_file_permission(file_path)

        for dir in dirs:
            dir_path = root_path / dir
            _set_folder_permission(dir_path)


def _get_mtime(year: int, month: int, day: int) -> float:
    """Get the mtime of a given date at midnight UTC."""
    date = datetime.datetime(year, month, day,
tzinfo=datetime.UTC)
    return time.mktime(date.timetuple())


# Map of relative paths, defined as regular expressions, to
expiration timestamps (mtime format).
# Partial matching will be used from top to bottom and the first
match will be chosen.
# Cached entries will be retained only if they are newer than the
expiration timestamp.
_INVALIDATE_CACHE_DIRS: dict[re.Pattern, float] = {
    re.compile("openpi-assets/checkpoints/pi0_aloha_pen_uncap"):
_get_mtime(2025, 2, 17),
    re.compile("openpi-assets/checkpoints/pi0_libero"):
_get_mtime(2025, 2, 6),
```

```
    re.compile("openpi-assets/checkpoints/"): _get_mtime(2025, 2,
3),
}


def _should_invalidate_cache(cache_dir: pathlib.Path, local_path:
pathlib.Path) -> bool:
    """Invalidate the cache if it is expired. Return True if the
cache was invalidated."""

    assert local_path.exists(), f"File not found at {local_path}"

    relative_path = str(local_path.relative_to(cache_dir))
    for pattern, expire_time in _INVALIDATE_CACHE_DIRS.items():
        if pattern.match(relative_path):
            # Remove if not newer than the expiration timestamp.
            return local_path.stat().st_mtime <= expire_time

    return False
```

四、安装适配的仿真环境 Libero

```
Plain Text
cd openpi
```

```
Plain Text
uv venv --python 3.8 examples/libero/.venv
```

该命令使用了 uv 工具在指定路径下创建一个 Python 虚拟环境（venv），并指定了 Python 版本为 3.8

```
Plain Text
source examples/libero/.venv/bin/activate
```

```
Plain Text
uv pip sync examples/libero/requirements.txt
third_party/libero/requirements.txt --extra-index-url
```

```
https://download.pytorch.org/whl/cu113 --index-strategy=unsafe-best-match
```

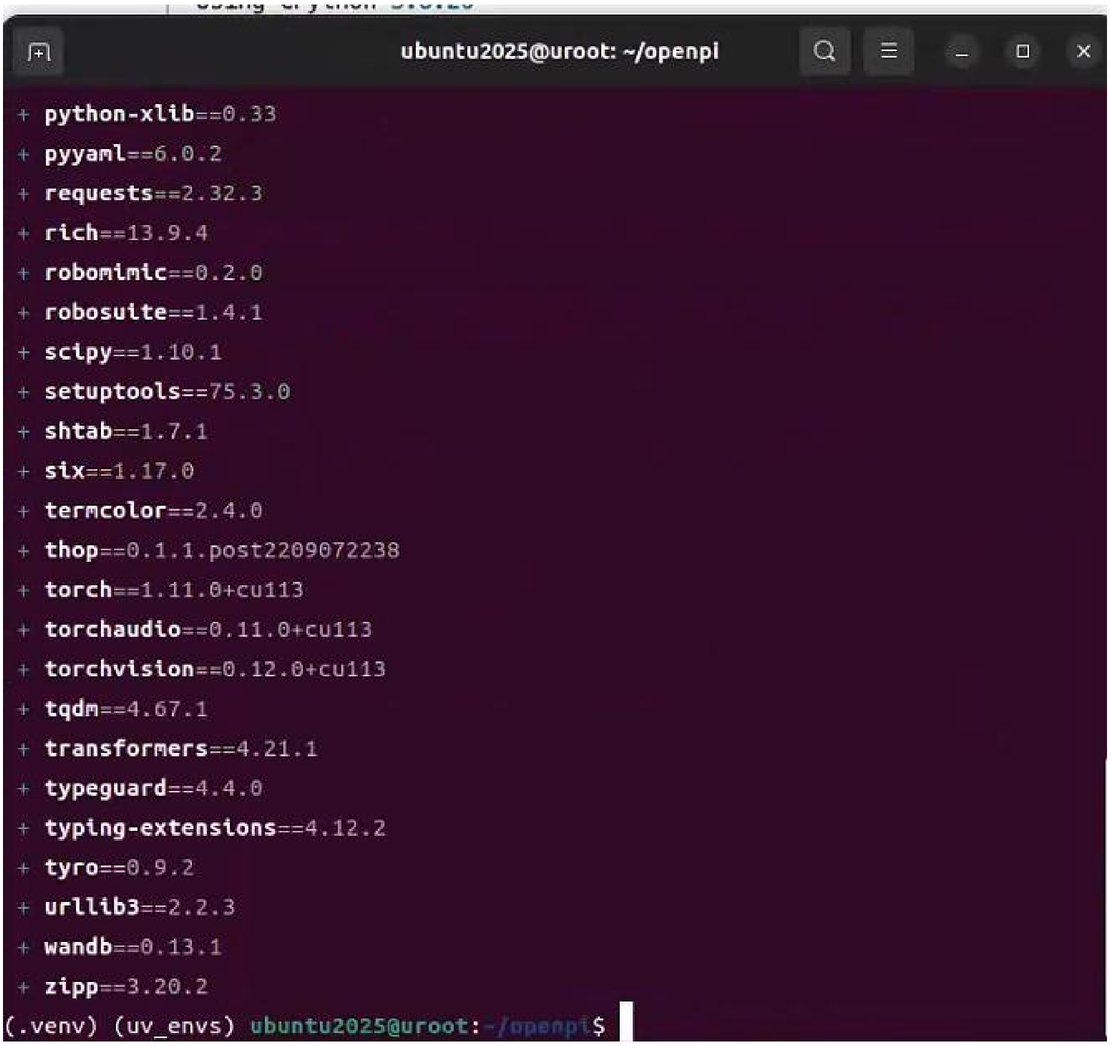
注意：这个步骤必须开代理，如果不会请参考上面第三步科学上网方法。

可能遇到的错误：

```
   Built evdev==1.7.1
   Built easydict==1.9
   Built robomimic==0.2.0
   Built bddl==1.0.1
   Built gym==0.25.2
   Built future==0.18.2
Preparing packages... (51/52)
Preparing packages... (51/52)
Preparing packages... (51/52)
  Failed to download `torch==1.11.0+cu113`
  ├─▶Failed to extract archive: torch-1.11.0+cu113-cp38-cp38-linux_x86_64.whl
  ├─▶I/O operation failed during extraction
  ╰─▶Failed to download distribution due to network timeout. Try increasing
     UV_HTTP_TIMEOUT (current value: 30s).
```

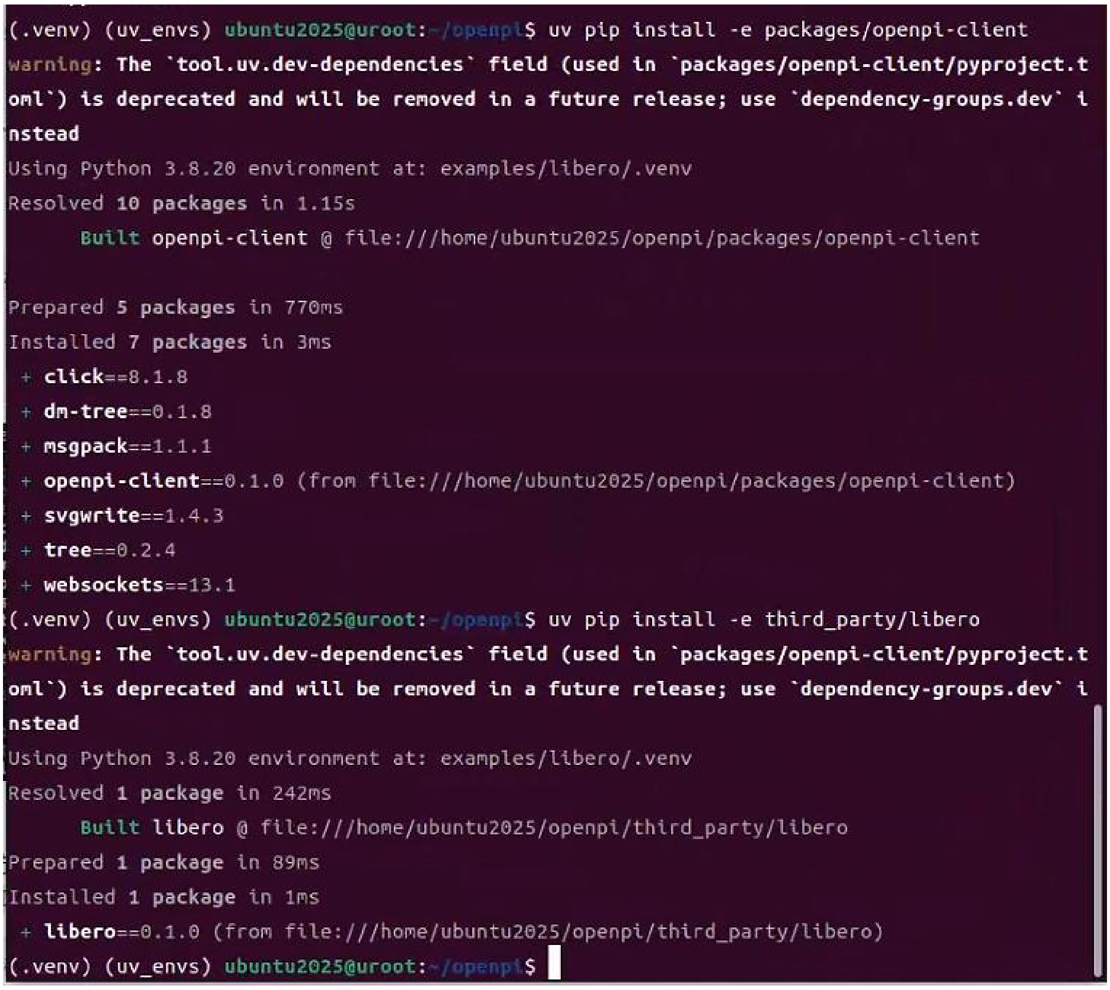
从错误信息来看，核心问题是下载 `torch==1.11.0+cu113` 时出现网络超时或提取失败，可能是由于网络不稳定、包体积较大（PyTorch GPU 版本通常通常超过 2GB）或超时时间过短导致的。

重新运行即可，正确结果如下：

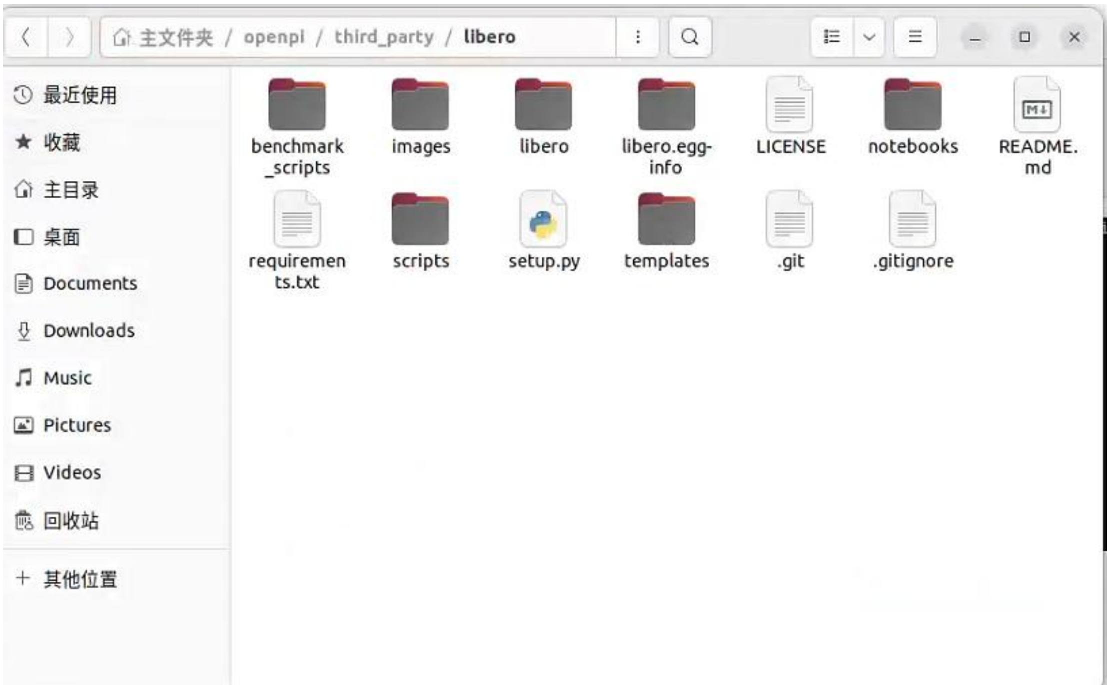


Plain Text
```
uv pip install -e packages/openpi-client
```

Plain Text
```
uv pip install -e third_party/libero
```



配置访问 libero 的环境变量时，要确保对应文件夹下存在“third_party/libero”，如果这里配置错误，会导致后续运行时找不到库 libero。



```
Plain Text
export PYTHONPATH=$PYTHONPATH:$PWD/third_party/libero
```

校验是否正常导入包

```
Plain Text
python -m libero
```

```
(.venv) (uv_envs) ubuntu2025@uroot:~/openpi$ export PYTHONPATH=$PYTHONPATH:$PWD/third_part
y/libero
(.venv) (uv_envs) ubuntu2025@uroot:~/openpi$ python -m libero
/home/ubuntu2025/openpi/examples/libero/.venv/bin/python: No module named libero.__main__;
 'libero' is a package and cannot be directly executed
(.venv) (uv_envs) ubuntu2025@uroot:~/openpi$ a
```

五、启动客户端

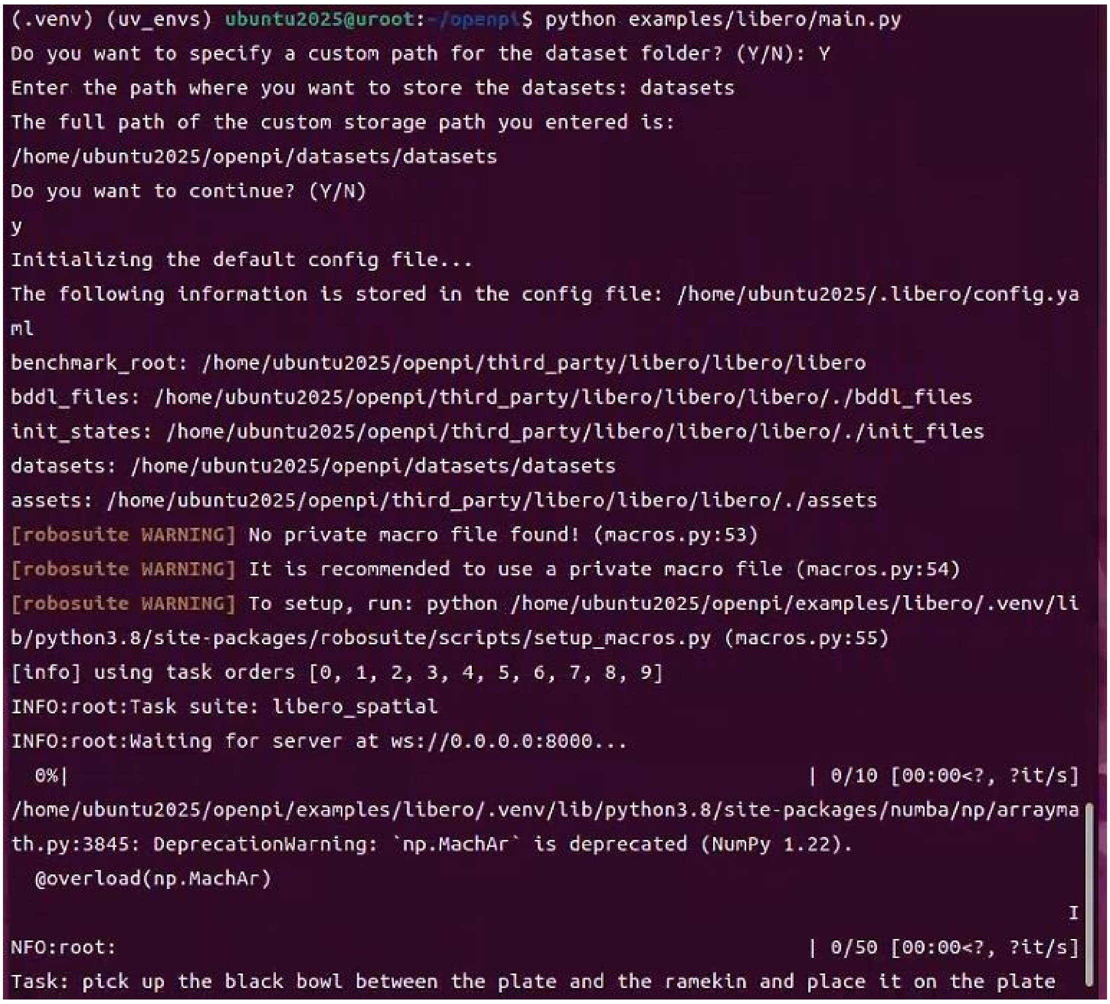
等待几分钟，具体由电脑算力影响。

```
Plain Text
cd openpi
conda activate uv_envs
source examples/libero/.venv/bin/activate
```

```
Plain Text
export MUJOCO_GL=glx
python examples/libero/main.py
```

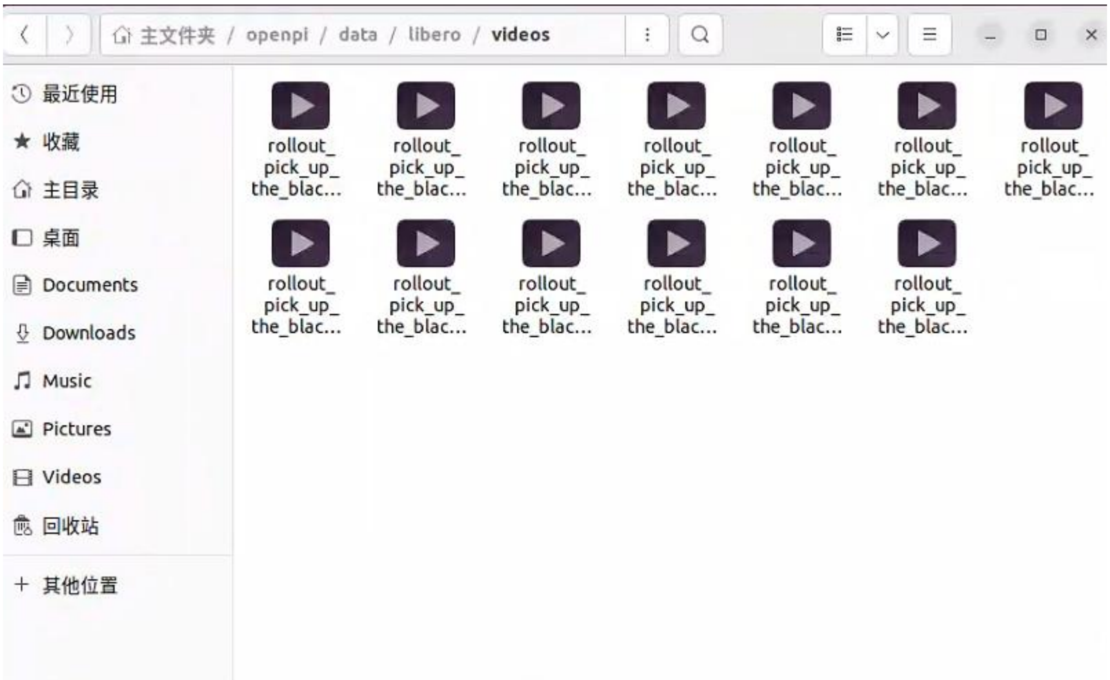
正常情况会看到下载数据集和执行的结果如下图所示：

```
(base) ubuntu2025@uroot:~$ conda activate uv_envs
(uv_envs) ubuntu2025@uroot:~$ cd openpi/ # 需要该目录下的：pyproject.toml
(uv_envs) ubuntu2025@uroot:~/openpi$ source examples/libero/.venv/bin/activate
(.venv) (uv_envs) ubuntu2025@uroot:~/openpi$ python examples/libero/main.py
```



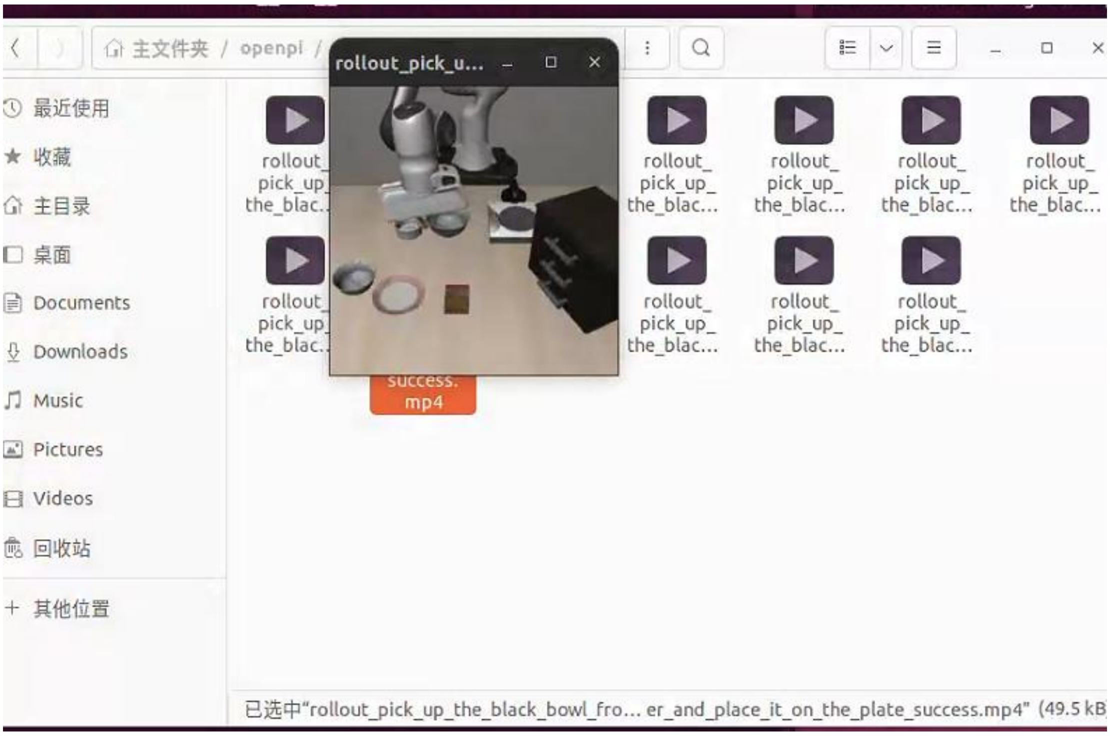
```
(.venv) (uv_envs) ubuntu2025@uroot:~/openpi$ python examples/libero/main.py
Do you want to specify a custom path for the dataset folder? (Y/N): Y
Enter the path where you want to store the datasets: datasets
The full path of the custom storage path you entered is:
/home/ubuntu2025/openpi/datasets/datasets
Do you want to continue? (Y/N)
y
Initializing the default config file...
The following information is stored in the config file: /home/ubuntu2025/.libero/config.ya
ml
benchmark_root: /home/ubuntu2025/openpi/third_party/libero/libero/libero
bddl_files: /home/ubuntu2025/openpi/third_party/libero/libero/libero/./bddl_files
init_states: /home/ubuntu2025/openpi/third_party/libero/libero/libero/./init_files
datasets: /home/ubuntu2025/openpi/datasets/datasets
assets: /home/ubuntu2025/openpi/third_party/libero/libero/libero/./assets
[robosuite WARNING] No private macro file found! (macros.py:53)
[robosuite WARNING] It is recommended to use a private macro file (macros.py:54)
[robosuite WARNING] To setup, run: python /home/ubuntu2025/openpi/examples/libero/.venv/li
b/python3.8/site-packages/robosuite/scripts/setup_macros.py (macros.py:55)
[info] using task orders [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
INFO:root:Task suite: libero_spatial
INFO:root:Waiting for server at ws://0.0.0.0:8000...
  0%|                                                         | 0/10 [00:00<?, ?it/s]
/home/ubuntu2025/openpi/examples/libero/.venv/lib/python3.8/site-packages/numba/np/arrayma
th.py:3845: DeprecationWarning: `np.MachAr` is deprecated (NumPy 1.22).
  @overload(np.MachAr)
                                                                                        I
NFO:root:                                                  | 0/50 [00:00<?, ?it/s]
Task: pick up the black bowl between the plate and the ramekin and place it on the plate
```

运行成功结果视频会保存在如下所示位置：



因为需要打开 mp4 格式视频，需要下载一个播放器：

```Plain Text
sudo apt-get install mpv
```



[rollout_pick_up_the_black_bowl_on_the_wooden_cabinet_and_place_it_on_the_plate_success.mp4]

可能遇到的错误：

这是 RoboSuite 在处理 EGL 渲染上下文时出现的问题，主要与图形渲染初始化或释放相关。

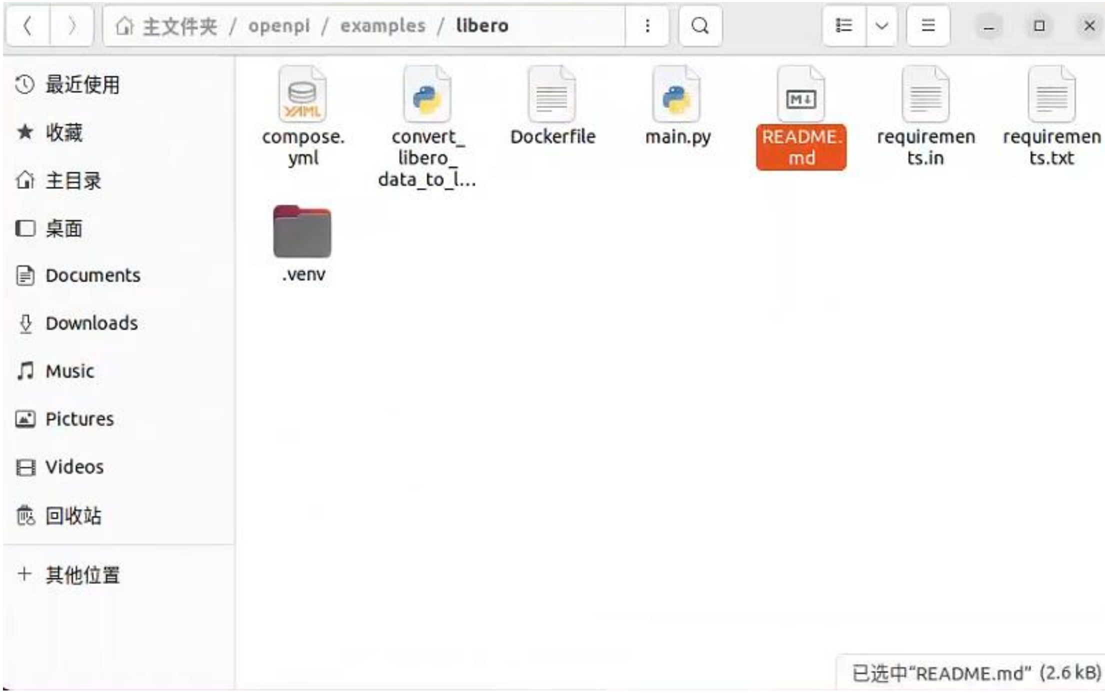
如果看到 EGL 错误，则可能需要安装以下依赖项：

```Plain Text
sudo apt-get install -y libegl1-mesa-dev libgles2-mesa-dev
```

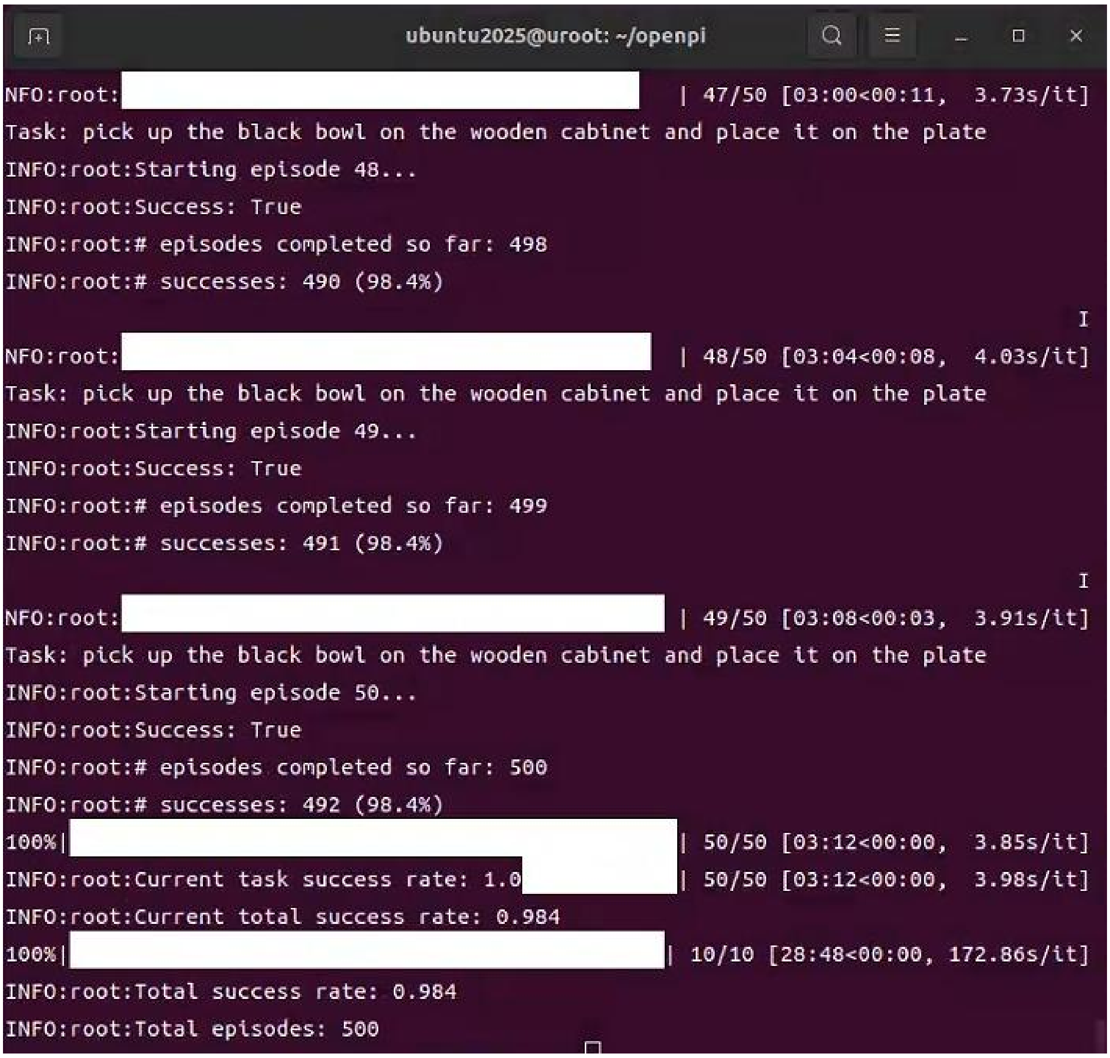
```
100%|████████████████████████████████████████| 10/10 [29:02<00:00, 174.29s/it]
INFO:root:Total success rate: 0.99
INFO:root:Total episodes: 500
Exception ignored in: <function MjRenderContext.__del__ at 0x7b27086b7670>
Traceback (most recent call last):
  File "/home/ubuntu2025/openpi/examples/libero/.venv/lib/python3.8/site-packages/robosuite/utils/binding_utils.py", line 199, in __del__
    self.gl_ctx.free()
  File "/home/ubuntu2025/openpi/examples/libero/.venv/lib/python3.8/site-packages/robosuite/renderers/context/egl_context.py", line 149, in free
    EGL.eglMakeCurrent(EGL_DISPLAY, EGL.EGL_NO_SURFACE, EGL.EGL_NO_SURFACE, EGL.EGL_NO_CONTEXT)
  File "/home/ubuntu2025/openpi/examples/libero/.venv/lib/python3.8/site-packages/OpenGL/error.py", line 230, in glCheckError
    raise self._errorClass(
OpenGL.raw.EGL._errors.EGLError: EGLError(
        err = EGL_NOT_INITIALIZED,
        baseOperation = eglMakeCurrent,
        cArguments = (
                <OpenGL._opaque.EGLDisplay_pointer object at 0x7b26faa579c0>,
                <OpenGL._opaque.EGLSurface_pointer object at 0x7b2715089b40>,
                <OpenGL._opaque.EGLSurface_pointer object at 0x7b2715089b40>,
                <OpenGL._opaque.EGLContext_pointer object at 0x7b2715089ac0>,
        ),
        result = 0
)
Exception ignored in: <function EGLGLContext.__del__ at 0x7b27086b74c0>
Traceback (most recent call last):
  File "/home/ubuntu2025/openpi/examples/libero/.venv/lib/python3.8/site-packages/robosuite/renderers/context/egl_context.py", line 155, in __del__
    self.free()
  File "/home/ubuntu2025/openpi/examples/libero/.venv/lib/python3.8/site-packages/robosuite/renderers/context/egl_context.py", line 149, in free
    EGL.eglMakeCurrent(EGL_DISPLAY, EGL.EGL_NO_SURFACE, EGL.EGL_NO_SURFACE, EGL.EGL_NO_CONTEXT)
  File "/home/ubuntu2025/openpi/examples/libero/.venv/lib/python3.8/site-packages/OpenGL/error.py", line 230, in glCheckError
    raise self._errorClass(
OpenGL.raw.EGL._errors.EGLError: EGLError(
        err = EGL_NOT_INITIALIZED,
        baseOperation = eglMakeCurrent,
        cArguments = (
                <OpenGL._opaque.EGLDisplay_pointer object at 0x7b26faa579c0>,
                <OpenGL._opaque.EGLSurface_pointer object at 0x7b2715089b40>,
                <OpenGL._opaque.EGLSurface_pointer object at 0x7b2715089b40>,
                <OpenGL._opaque.EGLContext_pointer object at 0x7b2715089ac0>,
```

参考官方文档第五十四行:

```
打开(O)    README.md  ~/openpi/examples/libero    保存(S)
37 ## Without Docker (not recommended)
38
39 Terminal window 1:
40
41 ```bash
42 # Create virtual environment
43 uv venv --python 3.8 examples/libero/.venv
44 source examples/libero/.venv/bin/activate
45 uv pip sync examples/libero/requirements.txt third_party/libero/requirements.txt --extra-index-url https://download.pytorch.org/whl/cu113 --index-strategy=unsafe-best-match
46 uv pip install -e packages/openpi-client
47 uv pip install -e third_party/libero
48 export PYTHONPATH=$PYTHONPATH:$PWD/third_party/libero
49
50 # Run the simulation
51 python examples/libero/main.py
52
53 # To run with glx for Mujoco instead (use this if you have egl errors):
54 MUJOCO_GL=glx python examples/libero/main.py
55 ```
56
57 Terminal window 2:
58
59 ```bash
60 # Run the server
61 uv run scripts/serve_policy.py --env LIBERO
62 ```
63
64 ## Results
65
66 If you want to reproduce the following numbers, you can evaluate the checkpoint at `gs://openpi-assets/checkpoints/pi05_libero/`. This
67 checkpoint was trained in openpi with the `pi05_libero` config.
68
69 | Model | Libero Spatial | Libero Object | Libero Goal | Libero 10 | Average |
70 |-------|---------------|---------------|-------------|-----------|---------|
71 | π0.5 @ 30k (finetuned) | 98.8 | 98.2 | 98.0 | 92.4 | 96.85
```




```
Plain Text
MUJOCO_GL=glx python examples/libero/main.py
```



至此，复现实验完全结束。

成功率：15 次抓取实验，5 次失败，成功率为 66.6%

二、微调实验

参考官方：https://huggingface.co/blog/pi0

LeRobot 在此 Hugging Face 社区页面上托管预训练模型和数据集：https://huggingface.co/lerobot

一、相关参数介绍

DROID：一个大规模的机器人数据集（Demonstrations for Robotic Imitation from Diverse sources），用来训练通用机器人策略。

ALOHA：一种低成本的双臂机器人系统（hardware setup），在 openpi 里有专门的 config，可以在模拟或真实硬件上跑。

在 openpi 中， pi05_droid 、 pi05_aloha 是配置的名字即 name， 就是针对不同平台的数据/ 配置的 policy 训练方案。后续迁移到自己的机械臂上可以自定义名字，例如利用so1o1 机械臂 pi05_so101、pi05_so101_lora。

已经微调好的模型分类：可直接推理——π0 DROID/π0 ALOHA/π0 Libero

π0 DROID：在 DROID 数据集上微调过的扩散π0

DROID 数据集由 Franka 机械臂在不同环境中执行的不同任务组成的开源数据集，且他们通过视频展示了 openpi 在训练数据中从未见过的环境中运行。其对应的 checkpoint 路径为：s3://openpi-assets/checkpoints/pi0_droid 推理速度比π0-FAST-DROID 快，但可能不遵循语言命令。

π0-FAST DROID：在 DROID 数据集微调过的π0-FAST

可以在 DROID 机器人平台上的新场景中执行各种简单的零样本桌面操控任务，例如“从烤面包机中取出面包”任务，其对应的 checkpoint 路径为：s3://openpi-assets/checkpoints/pi0_fast_droid

π0 ALOHA：含三套权重

根据 ALOHA(适合灵巧操作的低成本双臂系统) 数据进行了微调，可以在 ALOHA 机器人平台上进行毛巾折叠、食物舀取和其他任务，相当于提供了一套针对 ALOHA 平台上的任务进行微调的检查点[这些检查点可能对整体机器人设置非常敏感，但能够在完全未出现在训练数据中的全新 ALOHA 站点上运行它们]。

不同任务对应的不同 checkpoint 路径分别为：

折叠毛巾：s3://openpi-assets/checkpoints/pi0_aloha_towel

从容器中取出食物：s3://openpi-assets/checkpoints/pi0_aloha_tupperware

打开笔帽：s3://openpi-assets/checkpoints/pi0_aloha_pen_uncap

二、lerobot

克隆 Lerobot 仓库：

```
Plain Text
git clone https://github.com/huggingface/lerobot.git ~/lerobot
```

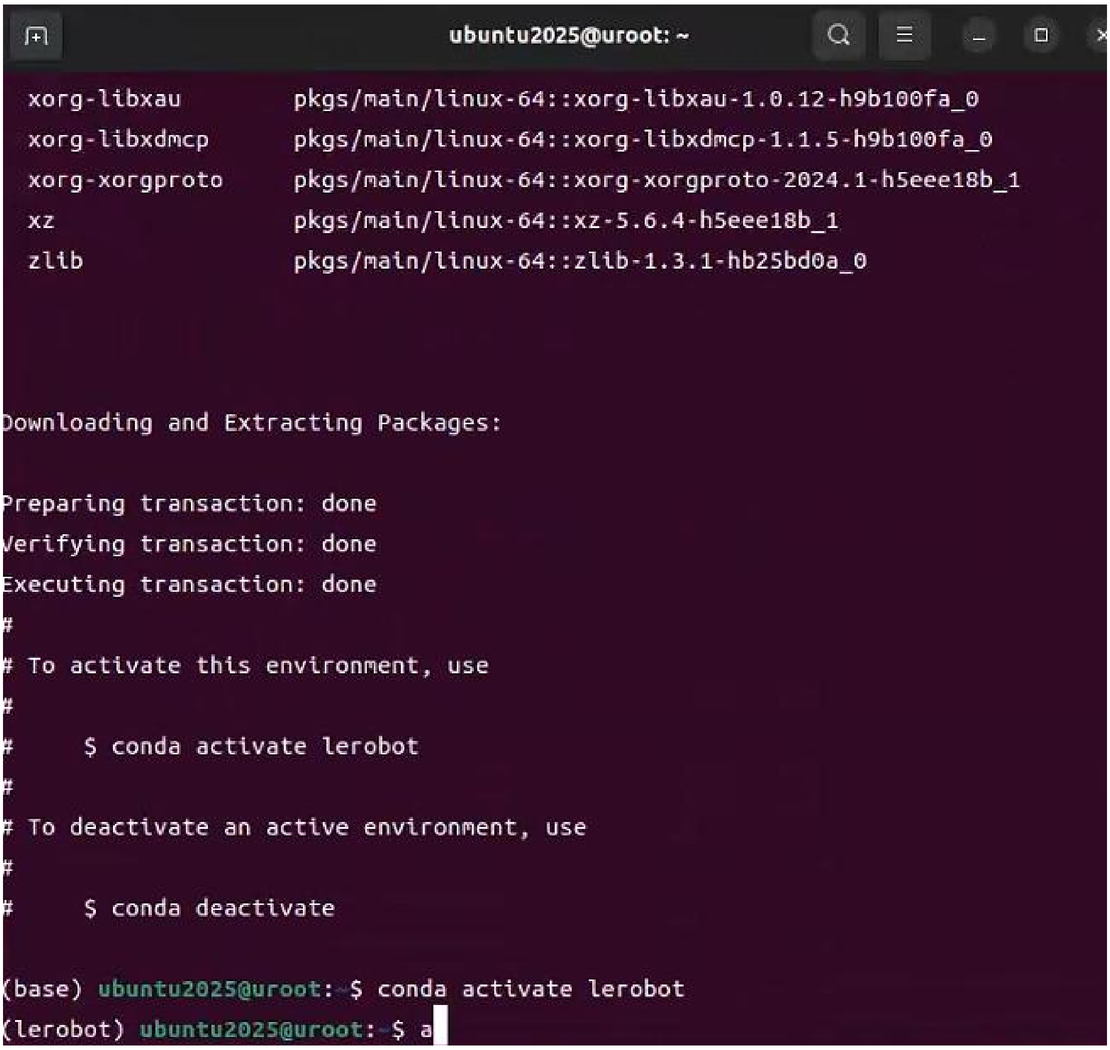
HTTPS 协议可能受网络限制，若 GitHub 账号配置了 SSH 密钥，可改用 SSH 克隆：

```
Plain Text
git clone git@github.com:huggingface/lerobot.git ~/lerobot
```

lerobot 环境设置

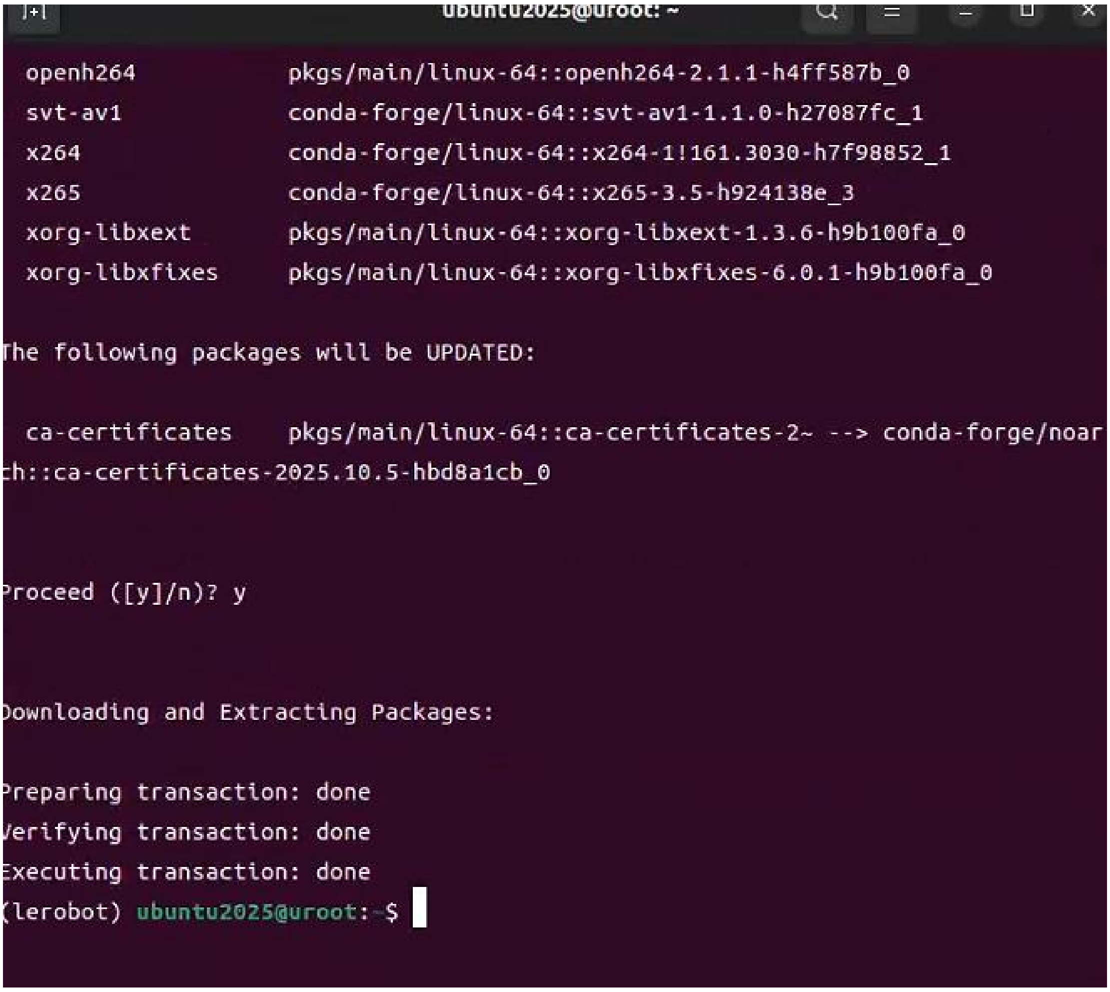
使用 Python 3.10 创建一个虚拟环境并激活它:

```
Plain Text
conda create -y -n lerobot python=3.10
conda activate lerobot
```

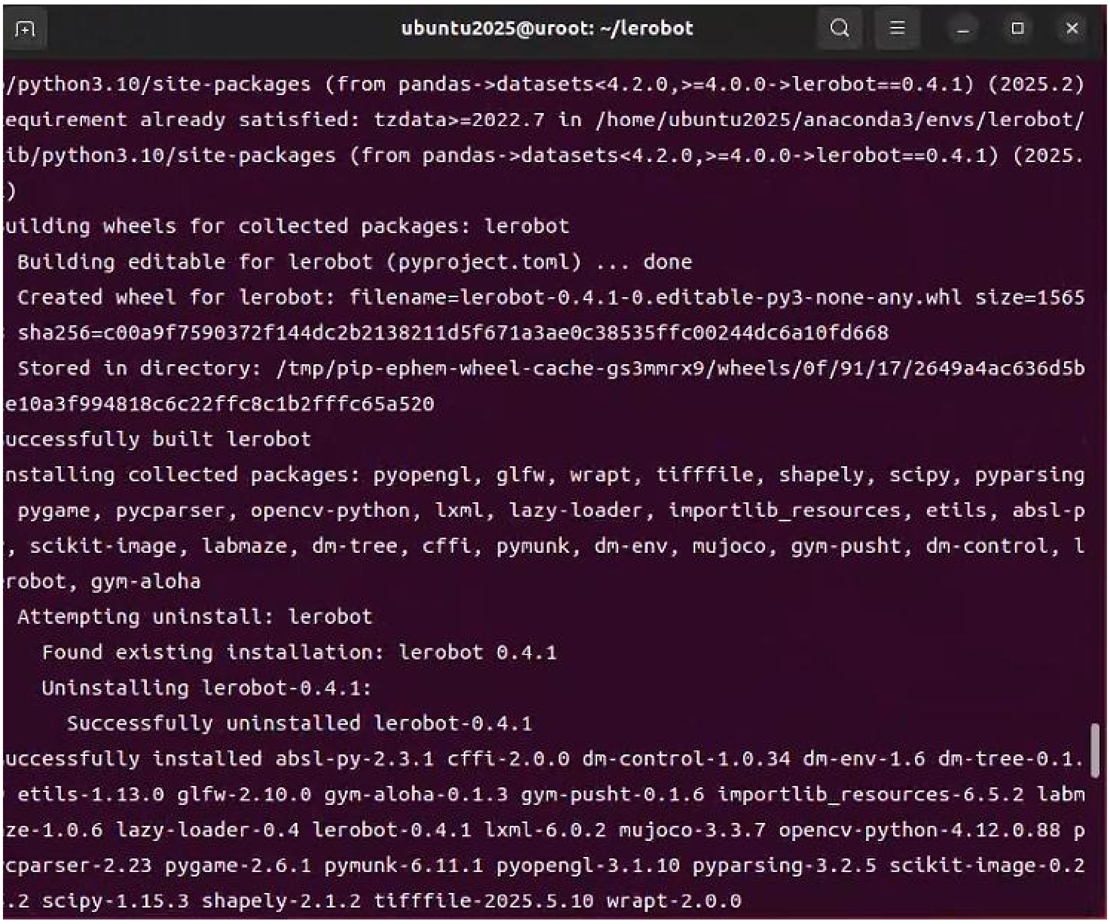


使用时，在您的环境中安装:

```
Plain Text
conda install ffmpeg -c conda-forge
```




```
Plain Text
cd lerobot
pip install -e . -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install -e ".[aloha, pusht]" -i
https://pypi.tuna.tsinghua.edu.cn/simple
```




```
Plain Text
python lerobot/src/lerobot/scripts/lerobot_eval.py \--
policy.path=/.cashe/openpi/openpi-assets/checkpoints/pi05_base
python lerobot/src/lerobot/scripts/eval.py \--
pretrained_policy.path=/path/to/pretrained/pi0
```

跑π0-FAST-DROID 模型的预训练检查点如下面代码所示:

```
Python
from openpi.training import config
from openpi.policies import policy_config
from openpi.shared import download

config = config.get_config("pi0_fast_droid")
checkpoint_dir = download.maybe_download("s3://openpi-
assets/checkpoints/pi0_fast_droid")

# Create a trained policy.
policy = policy_config.create_trained_policy(config,
checkpoint_dir)
```

```
# Run inference on a dummy example.
example = {
    "observation/exterior_image_1_left": ...,
    "observation/wrist_image_left": ...,
    ...
    "prompt": "pick up the fork"
}
action_chunk = policy.infer(example)["actions"]
```

三、微调 π0 预训练模型

要使用 中的检查点微调 π0 模型，请运行以下命令：

```Plain Text
python lerobot/scripts/train.py \--policy.path=lerobot/pi0 \--dataset.repo_id=danaaubakirova/koch_test
```

要使用 PaliGemma 和 Expert Gemma 微调 π0 神经网络（在 π0 微调之前使用 VLM 默认参数进行预训练），请执行：

```Plain Text
python lerobot/scripts/train.py \--policy.type=pi0 \--dataset.repo_id=danaaubakirova/koch_test
```

您还可以使用以下代码独立于 LeRobot 训练框架使用预训练的 π0 模型：

```Plain Text
policy = Pi0Policy.from_pretrained("lerobot/pi0")
```

四、基于自己的数据集微调π0 基础模型

利用 Libero 数据集微调π0 基础模型的三个步骤

第一步，将 Libero 数据集转换为 LeRobot 数据集 v2.0 格式：

官方在 [examples/libero/convert_libero_data_to_lerobot.py] 中提供了一个最小的示例脚本，用于将 LIBERO 数据转换为 LeRobot 数据集。您可以轻松修改它以转换您自己的数据！

首先下载原始 Libero 数据集（https://huggingface.co/datasets/openvla/modified_libero_rlds），然后使用以下命令运行脚本：

```
Plain Text
uv run examples/libero/convert_libero_data_to_lerobot.py --
data_dir /path/to/your/libero/data
```

第二步，定义使用自定义数据集的训练配置，并运行训练：

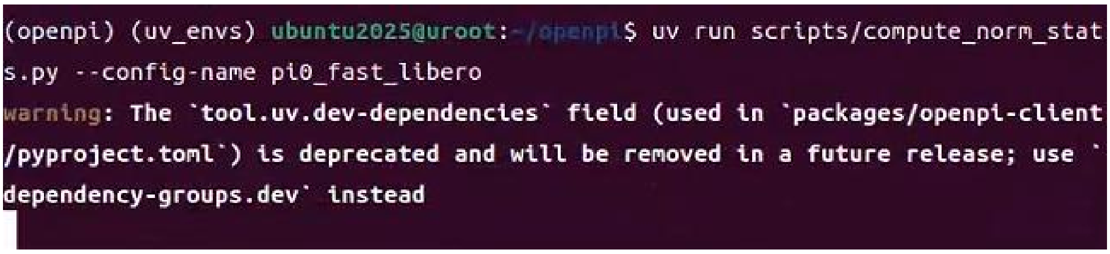
若要根据自己的数据微调基础模型，需要定义数据处理和训练的配置。官方在下面提供了带有 LIBERO 详细注释的示例配置，您可以针对自己的数据集进行修改：

- LiberoInputs 和 LiberoOutputs：定义从 LIBERO 环境到模型的数据映射，反之亦然。将用于训练和推理。
- LeRobotLiberoDataConfig：定义如何处理 LeRobot 数据集中的原始 LIBERO 数据以进行训练。
- TrainConfig：定义微调超参数、数据配置和权重加载器。

官方提供了 LIBERO 数据上的 π_0、π_0-FAST 和 $\pi_{0.5}$ 的微调配置示例。

在运行训练之前，可以计算训练数据的标准化统计数据，使用训练配置的名称运行以下脚本：

```
Plain Text
uv run scripts/compute_norm_stats.py --config-name pi0_fast_libero
```



```
WARNING:root:
The dataset you requested (physical-intelligence/libero) is in 2.0 format.
While current version of LeRobot is backward-compatible with it, the version o
f your dataset still uses global
stats instead of per-episode stats. Update your dataset stats to the new forma
t using this command:
```
python lerobot/common/datasets/v21/convert_dataset_v20_to_v21.py --repo-id=phy
sical-intelligence/libero
```

If you encounter a problem, contact LeRobot maintainers on [Discord](https://d
iscord.com/invite/s3KuuzsPFb)
or open an [issue on GitHub](https://github.com/huggingface/lerobot/issues/new
/choose).

Resolving data files: 100%|          | 1693/1693 [00:00<00:00, 87523.50it/s]
Loading dataset shards: 100%|          | 70/70 [00:00<00:00, 3048.47it/s]
Computing stats: 100%|          | 8545/8545 [20:17<00:00,  7.02it/s]
Writing stats to: /home/ubuntu2025/openpi/assets/pi0_fast_libero/physical-inte
lligence/libero
(openpi) (uv_envs) ubuntu2025@uroot:~/openpi$
```

接下来，可以使用以下命令开始训练（`--overwrite` 如果使用相同的配置重新运行微调，则该标志用于覆盖现有检查点)

特别注意：这一步需要科学上网，否则无法连接到 Hugging Face 的服务器（`huggingface.co`），导致模型的 tokenizer/processor 加载失败（超时错误）。

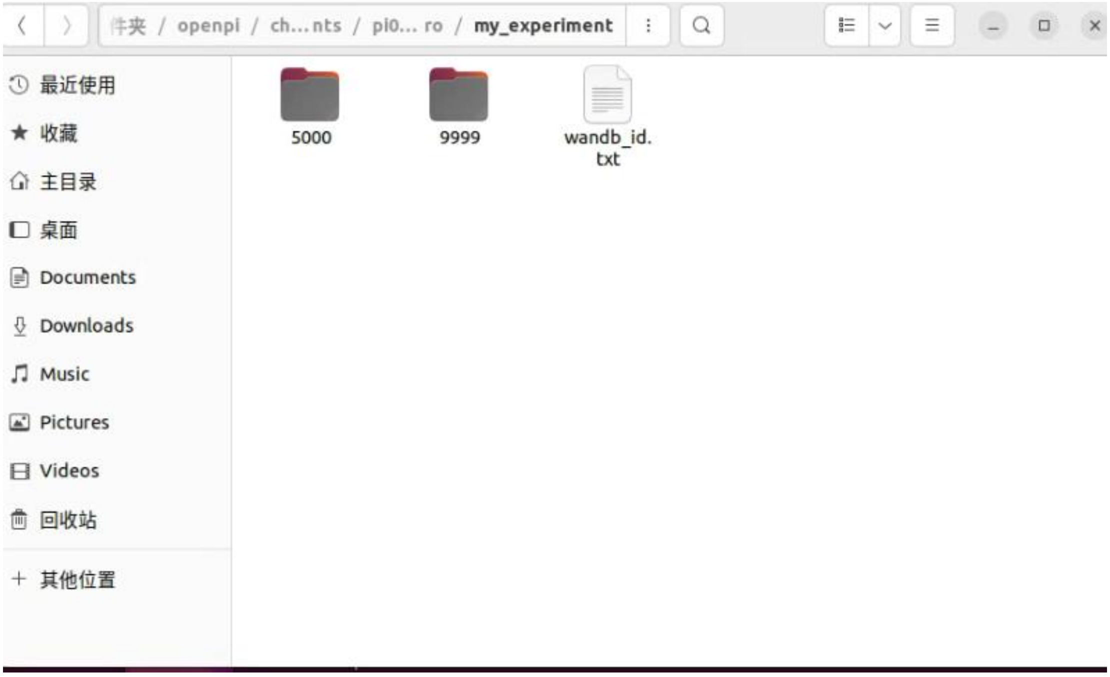
```
Plain Text
XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run scripts/train.py
pi0_fast_libero --exp-name=my_experiment --overwrite
```

该命令会将训练进度记录到控制台并将检查点保存到 checkpoints 目录中。还可以在 Weights & Biases 仪表板上监控训练进度。为了最大限度地利用 GPU 内存，请 XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 在运行训练之前设置 - 这使 JAX 能够使用高达 90% 的 GPU 内存（而默认值为 75%）。

```
:170)
20:20:46.381 [I] Found 0 checkpoint steps in /home/ubuntu2025/openpi/checkpo
ts/pi0_fast_libero/my_experiment (1735727:checkpoint_manager.py:1701)
20:20:46.381 [I] [process=0][thread=MainThread] CheckpointManager created,
imary_host=0, CheckpointManagerOptions=CheckpointManagerOptions(save_interva
steps=1, max_to_keep=1, keep_time_interval=None, keep_period=5000, should_ke
_fn=None, best_fn=None, best_mode='max', keep_checkpoints_without_metrics=Tr
, step_prefix=None, step_format_fixed_length=None, step_name_format=None, cr
te=False, cleanup_tmp_directories=False, save_on_steps=frozenset(), single_h
t_load_and_broadcast=False, todelete_subdir=None, enable_background_delete=F
se, read_only=False, enable_async_checkpointing=True, async_options=AsyncOpt
ns(timeout_secs=7200, barrier_sync_fn=None, post_finalization_callback=None,
reate_directories_asynchronously=True), multiprocessing_options=Multiprocess
gOptions(primary_host=0, active_processes=None, barrier_sync_key_prefix=None
 should_save_fn=None, file_options=FileOptions(path_permission_mode=None), s
e_root_metadata=True, temporary_path_class=None, save_decision_policy=None,
event_write_metrics=False), root_directory=/home/ubuntu2025/openpi/checkpoin
/pi0_fast_libero/my_experiment: <orbax.checkpoint.checkpoint_manager.Checkpo
tManager object at 0x767d0b411f10> (1735727:checkpoint_manager.py:801)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice:
```

训练完成后会生成一些文件，分别保存在以下路径

- 核心模型数据：`/home/ubuntu2025/openpi/checkpoints/pi0_fast_libero/my_experiment/`
- 训练日志数据：`/home/ubuntu2025/openpi/wandb/`



根据你的需求选择即可，以下是各选项的建议：

- 若你需要跟踪训练过程（如查看损失曲线、参数变化等），且已有 Weights & Biases（wandb）账号：选择 2，然后输入你的 wandb API 密钥（可在 wandb 官网个人设置中获取）。
- 若你没有 wandb 账号但想使用可视化功能：选择 1，按提示注册账号并获取 API 密钥，完成登录后即可跟踪训练。
- 若你暂时不需要可视化，只想尽快运行训练脚本：选择 3，脚本会跳过 wandb 初始化，直接进入训练流程。

可能出现以下报错：

```
    return wrapped_fn_impl(args, kwargs, bound, memos)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ubuntu2025/openpi/.venv/lib/python3.11/site-packages/jaxtyping/_
decorator.py", line 483, in wrapped_fn_impl
    out = fn(*args, **kwargs)
          ^^^^^^^^^^^^^^^^^^^
  File "/home/ubuntu2025/openpi/scripts/train.py", line 126, in init_train_sta
te
    train_state = jax.jit(
                  ^^^^^^^^
jaxlib.xla_extension.XlaRuntimeError: RESOURCE_EXHAUSTED: Out of memory while
trying to allocate 2415919104 bytes.
--------------------
For simplicity, JAX has removed its internal frames from the traceback of the
following exception. Set JAX_TRACEBACK_FILTERING=off to include these.
wandb:
wandb: You can sync this run to the cloud by running:
wandb: wandb sync /home/ubuntu2025/openpi/wandb/offline-run-20251101_204757-3k
p1l2i0
wandb: Find logs at: wandb/offline-run-20251101_204757-3kp1l2i0/logs
(openpi) (uv_envs) ubuntu2025@uroot:~/openpi$
```

模型初始化时的内存需求过大（43.56GiB）导致 GPU 内存不足。需要从模型参数精度、输入数据尺寸、内存分配策略三方面深度优化，具体修改 train.py 内容如下：

1、调整 `max_safe_batch_size`（依次按照 2 的倍数往下降）

```python
def main(config: _config.TrainConfig):
    init_logging()
    logging.info(f"Running on: {platform.node()}")
    # 新增：根据设备内存动态减小批次大小（示例值，根据实际显存调整）
    max_safe_batch_size = 8  # 16GB GPU 建议值，8GB 可设为 8
    if config.batch_size > max_safe_batch_size:
        logging.warning(f"Batch size {config.batch_size} is too large. Reducing to {max_safe_batch_size}")
        config = dataclasses.replace(config, batch_size=max_safe_batch_size)

    # 原批次检查逻辑
    if config.batch_size % jax.device_count() != 0:
        raise ValueError(
            f"Batch size {config.batch_size} must be divisible by the number of devices {jax.device_count()}."
        )
    if config.batch_size % jax.device_count() != 0:
        raise ValueError(
            f"Batch size {config.batch_size} must be divisible by the number of devices {jax.device_count()}."
        )

    jax.config.update("jax_compilation_cache_dir", str(epath.Path("~/.cache/jax").expanduser()))

    rng = jax.random.key(config.seed)
    train_rng, init_rng = jax.random.split(rng)

    mesh = sharding.make_mesh(config.fsdp_devices)
    data_sharding = jax.sharding.NamedSharding(mesh, jax.sharding.PartitionSpec(sharding.DATA_AXIS))
    replicated_sharding = jax.sharding.NamedSharding(mesh, jax.sharding.PartitionSpec())

    checkpoint_manager, resuming = _checkpoints.initialize_checkpoint_dir(
        config.checkpoint_dir,
        keep_period=config.keep_period,
        overwrite=config.overwrite,
        resume=config.resume,
```

2、`init_train_state` 函数中参数精度转换

```python
@at.typecheck
def init_train_state(
    config: _config.TrainConfig, init_rng: at.KeyArrayLike, mesh: jax.sharding.Mesh, *, resume: bool
) -> tuple[training_utils.TrainState, Any]:
    tx = _optimizer.create_optimizer(config.optimizer, config.lr_schedule, weight_decay_mask=None)

    def init(rng: at.KeyArrayLike, partial_params: at.Params | None = None) -> training_utils.TrainState:
        rng, model_rng = jax.random.split(rng)
        # initialize the model (and its parameters).
        model = config.model.create(model_rng)

        # Merge the partial params into the model.
        if partial_params is not None:
            graphdef, state = nnx.split(model)
            # This will produce an error if the partial params are not a subset of the state.
            state.replace_by_pure_dict(partial_params)
            model = nnx.merge(graphdef, state)

        params = nnx.state(model)
        # Convert frozen params to bfloat16.
        #params = nnx_utils.state_map(params, config.freeze_filter, lambda p: p.replace(p.value.astype(jnp.bfloat16)))
        params = nnx_utils.state_map(params, lambda path, value: True, lambda p: p.replace(p.value.astype(jnp.bfloat16)))
        return training_utils.TrainState(
            step=0,
            params=params,
            model_def=nnx.graphdef(model),
            tx=tx,
            opt_state=tx.init(params.filter(config.trainable_filter)),
            ema_decay=config.ema_decay,
            ema_params=None if config.ema_decay is None else params,
        )
```

3、禁用图像日志（减少内存消耗）

即使选择不使用 wandb 可视化，图像日志生成仍会占用大量内存，需彻底注释。

```python
)
data_iter = iter(data_loader)
batch = next(data_iter)
logging.info(f"Initialized data loader:\n{training_utils.array_tree_to_info(batch)}")

# Log images from first batch to sanity check.
#images_to_log = [
#    wandb.Image(np.concatenate([np.array(img[i]) for img in batch[0].images.values()], axis=1))
#    for i in range(min(5, len(next(iter(batch[0].images.values())))))
# ]
# wandb.log({"camera_views": images_to_log}, step=0)
# 新增：确认禁用图像日志
logging.info("Image logging disabled to reduce memory usage")

train_state, train_state_sharding = init_train_state(config, init_rng, mesh, resume=resuming)
jax.block_until_ready(train_state)
logging.info(f"Initialized train state:\n{training_utils.array_tree_to_info(train_state.params)}")
```

4、修复日志格式化错误（强制转换数值类型）

在生成 `info_str` 时，将 v 强制转换为 `float`，确保即使 v 是字符串形式的数字也能正常格式化。

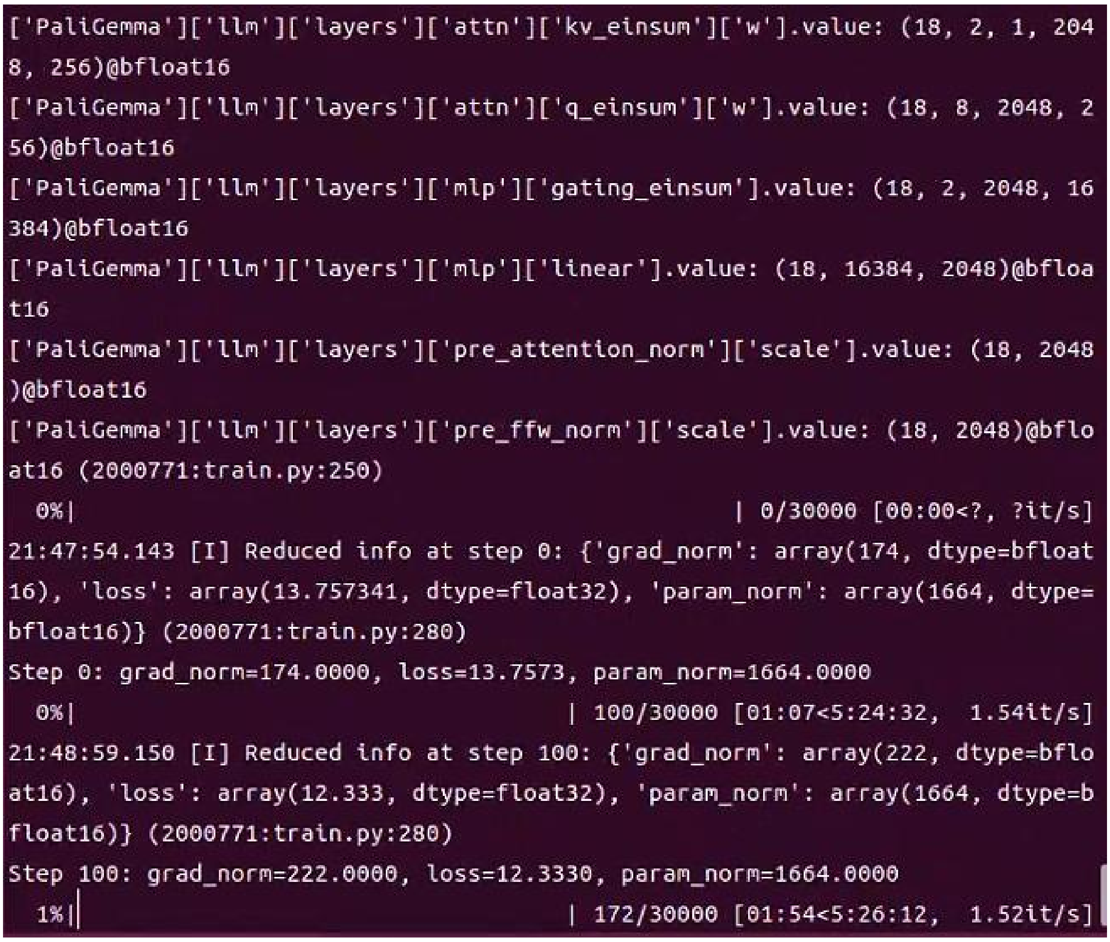
```python
infos = []
for step in pbar:
    with sharding.set_mesh(mesh):
        train_state, info = ptrain_step(train_rng, train_state, batch)
    infos.append(info)
    if step % config.log_interval == 0:
        stacked_infos = common_utils.stack_forest(infos)
        reduced_info = jax.device_get(jax.tree.map(jnp.mean, stacked_infos))

        # 新增：打印reduced_info内容，排查异常值（调试用）
        logging.info(f"Reduced info at step {step}: {reduced_info}")

        # 修复：强制转换v为float
        info_str = ", ".join(f"{k}={float(v):.4f}" for k, v in reduced_info.items())

        #info_str = ", ".join(f"{k}={v:.4f}" for k, v in reduced_info.items())
        pbar.write(f"Step {step}: {info_str}")
        wandb.log(reduced_info, step=step)
        infos = []
    batch = next(data_iter)
```

最后训练成功如下图所示:



```
['PaliGemma']['llm']['layers']['attn']['kv_einsum']['w'].value: (18, 2, 1, 204
8, 256)@bfloat16
['PaliGemma']['llm']['layers']['attn']['q_einsum']['w'].value: (18, 8, 2048, 2
56)@bfloat16
['PaliGemma']['llm']['layers']['mlp']['gating_einsum'].value: (18, 2, 2048, 16
384)@bfloat16
['PaliGemma']['llm']['layers']['mlp']['linear'].value: (18, 16384, 2048)@bfloa
t16
['PaliGemma']['llm']['layers']['pre_attention_norm']['scale'].value: (18, 2048
)@bfloat16
['PaliGemma']['llm']['layers']['pre_ffw_norm']['scale'].value: (18, 2048)@bflo
at16 (2000771:train.py:250)
  0%|                                              | 0/30000 [00:00<?, ?it/s]
21:47:54.143 [I] Reduced info at step 0: {'grad_norm': array(174, dtype=bfloat
16), 'loss': array(13.757341, dtype=float32), 'param_norm': array(1664, dtype=
bfloat16)} (2000771:train.py:280)
Step 0: grad_norm=174.0000, loss=13.7573, param_norm=1664.0000
  0%|                                  | 100/30000 [01:07<5:24:32,  1.54it/s]
21:48:59.150 [I] Reduced info at step 100: {'grad_norm': array(222, dtype=bflo
at16), 'loss': array(12.333, dtype=float32), 'param_norm': array(1664, dtype=b
float16)} (2000771:train.py:280)
Step 100: grad_norm=222.0000, loss=12.3330, param_norm=1664.0000
  1%||                                  | 172/30000 [01:54<5:26:12,  1.52it/s]
```

[6edc2b1277ceb81d970d21e94c77b035.mp4]

5、修改训练轮次，防止电脑内存不足（可选）

```
Plain Text
XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run scripts/train.py
pi0_fast_libero --exp-name=my_experiment --overwrite --
num_train_steps 10000
```

```
['PaliGemma']['img']['head']['bias'].value: (2048,)@bfloat16
['PaliGemma']['img']['head']['kernel'].value: (1152, 2048)@bfloat16
['PaliGemma']['img']['pos_embedding'].value: (1, 256, 1152)@bfloat16
['PaliGemma']['llm']['embedder']['input_embedding'].value: (257152, 2048)@bfloat
16
['PaliGemma']['llm']['final_norm']['scale'].value: (2048,)@bfloat16
['PaliGemma']['llm']['layers']['attn']['attn_vec_einsum']['w'].value: (18, 8, 25
6, 2048)@bfloat16
['PaliGemma']['llm']['layers']['attn']['kv_einsum']['w'].value: (18, 2, 1, 2048,
 256)@bfloat16
['PaliGemma']['llm']['layers']['attn']['q_einsum']['w'].value: (18, 8, 2048, 256
)@bfloat16
['PaliGemma']['llm']['layers']['mlp']['gating_einsum'].value: (18, 2, 2048, 1638
4)@bfloat16
['PaliGemma']['llm']['layers']['mlp']['linear'].value: (18, 16384, 2048)@bfloat1
6
['PaliGemma']['llm']['layers']['pre_attention_norm']['scale'].value: (18, 2048)@
bfloat16
['PaliGemma']['llm']['layers']['pre_ffw_norm']['scale'].value: (18, 2048)@bfloat
16 (15977:train.py:252)
  0%|                                                      | 0/10000 [00:00<?, ?it/s]
16:35:30.836 [I] Reduced info at step 0: {'grad_norm': array(174, dtype=bfloat16
), 'loss': array(13.757341, dtype=float32), 'param_norm': array(1664, dtype=bflo
at16)} (15977:train.py:282)
Step 0: grad_norm=174.0000, loss=13.7573, param_norm=1664.0000
  0%|                                           | 16/10000 [00:13<1:48:08,  1.54it/s]
```

第三步，启动策略服务器并运行推理：

训练完成后，可以通过启动策略服务器，然后从 Libero 评估脚本中查询它来运行推理。启动模型服务器很容易「在此示例中使用迭代 9999 的检查点，但可以根据需要进行修改」

```
Plain Text
uv run scripts/serve_policy.py policy:checkpoint --
policy.config=pi0_fast_libero --
policy.dir=checkpoints/pi0_fast_libero/my_experiment/9999
```

这将启动一个侦听端口 8000 的服务器并等待将观察结果发送到它。然后，我们可以运行查询服务器的评估脚本（或机器人运行时）。

```
INFO:2025-11-02 18:47:57,527:jax._src.xla_bridge:925: Unable to initialize backe
nd 'tpu': INTERNAL: Failed to open libtpu.so: libtpu.so: cannot open shared obje
ct file: No such file or directory
INFO:jax._src.xla_bridge:Unable to initialize backend 'tpu': INTERNAL: Failed to
 open libtpu.so: libtpu.so: cannot open shared object file: No such file or dire
ctory
INFO:absl:orbax-checkpoint version: 0.11.13
INFO:absl:Created BasePyTreeCheckpointHandler: use_ocdbt=True, use_zarr3=False,
pytree_metadata_options=PyTreeMetadataOptions(support_rich_types=False), array_m
etadata_store=<orbax.checkpoint._src.metadata.array_metadata_store.Store object
at 0x70065ea8a9d0>
INFO:absl:Restoring checkpoint from /home/ubuntu2025/openpi/checkpoints/pi0_fast
_libero/my_experiment/9999/params.
INFO:absl:[thread=MainThread] Failed to get flag value for EXPERIMENTAL_ORBAX_US
E_DISTRIBUTED_PROCESS_ID.
INFO:absl:[process=0] /jax/checkpoint/read/bytes_per_sec: 2.2 GiB/s (total bytes
: 5.4 GiB) (time elapsed: 2 seconds) (per-host)
INFO:absl:Finished restoring checkpoint in 2.50 seconds from /home/ubuntu2025/op
enpi/checkpoints/pi0_fast_libero/my_experiment/9999/params.
INFO:root:Loaded norm stats from /home/ubuntu2025/openpi/assets/pi0_fast_libero/
physical-intelligence/libero
INFO:root:Loaded norm stats from /home/ubuntu2025/openpi/checkpoints/pi0_fast_li
bero/my_experiment/9999/assets/physical-intelligence/libero
INFO:root:Creating server (host: uroot, ip: 127.0.1.1)
INFO:websockets.server:server listening on 0.0.0.0:8000
```